



The Open University

M263 Building Blocks of Software

Unit 14

Proof

Study guide

This unit contains four sections, of which the first is a short and informal introduction to the main technique discussed in the unit. The most important teaching material is covered in Sections 2 and 3. These two sections are of about equal length and importance. Section 2 develops the technique in a mathematical setting, one that is used in Unit 15, whereas Section 3 uses the technique in a software setting. Section 4, which is quite short, describes a few further applications of the technique.

There are a few practical computer activities associated with Section 3. You should spend no more than about 5% of your study time for this unit on practical computing.

Some of the proofs involve manipulative algebra. If you do not have much experience with this, do not spend too long trying to follow the details of the algebra. Subsection 2.4 is “read only”, and will not be assessed.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenvq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 6839 8

Contents

Introduction	4
1 Mathematical induction	6
2 Proofs about recurrence systems	8
2.1 The principle of mathematical induction	8
2.2 Proving formulae for recurrence systems	11
2.3 Formulae for sums	16
2.4 Extending the principle of induction	17
3 Proofs about algorithms involving while loops	20
3.1 A loop invariant condition	20
3.2 Proofs of correctness	26
4 Other proofs using induction	36
Summary of Unit 14	40
Solutions to the exercises	41
Index	47

Introduction

In mathematics, a **proof** is a formalised argument that is designed to convince the reader, beyond doubt, of the truth of some assertion — a theorem. A mathematical proof starts with known facts which are turned, by means of agreed rules of deduction, into new information. This process often follows a well understood structure.

Computing has its origins in mathematics and so it is natural that mathematical formalism and its conventions, including proof, feature in most texts dealing with the theory of computing. This said, it must be acknowledged that for some considerable time there has been debate, not to say controversy, over the wider role of formalism and proof in computing. In particular, there has been disagreement over its place in programming and software engineering. On the one hand, there are those who maintain that a software engineer who claims that a specification for a system meets its requirements, or a programmer who claims that some code performs the required computation, should be able to prove formally their respective assertions. On the other hand, there are those who maintain that this view is utopian. They offer various arguments to support their position. For example, they point out that a formal proof, often many times longer than the original code, is as likely to contain an error as the code itself — especially if it is produced by the same individual! And let us not forget that systems can easily contain many thousands of lines of code, much of which may be legacy code (i.e. code inherited from older systems), which is poorly documented or understood, let alone proven.

Another objection is that proving a program correct is of little value if the specification that the program satisfies is inappropriate in the first place — all you get is a correct solution to the wrong problem. Shifting the emphasis to formalising the system's specification, and proving that correct, does not really help matters if the formal specification does not accurately or entirely capture the requirements of the client.

It is clear that the role of formal proof in computing differs from that in mathematics. An important observation is that proof in programming and software engineering involves a different social process than in mathematics. When a mathematician produces a proof he or she will show it to colleagues who will often spot errors, which might be major or minor. Assuming these errors can be corrected, the proof will usually be presented to a conference audience or submitted to a peer-reviewed journal — possibly both. Here again it is closely scrutinised by other mathematicians, and again, errors or weaknesses are often highlighted and then corrected before final publication. Once published, the proof is seen by a yet larger audience. This process plays an important part in ensuring that a mathematical proof is robust. Whilst proofs in fundamental computer science undergo a similar process, this is rarely the case in day-to-day programming and software engineering. Thus the scope for undetected error remains very high.

If the two sides of this argument meet, it is perhaps to agree that requiring proof of specifications or programs is a useful additional quality assurance process if the consequences of software being in error are such that the additional effort is warranted. Examples include contexts in which lives might be lost or significant economic repercussions ensue. A useful approach in such projects is to use a variation on what is known as a clean room. A **clean room** attempts to reproduce some of the elements of social processes

You saw some examples of proofs, where the agreed rules of deduction were clearly specified, in Unit 10. In general, mathematical proofs do not specify the rules of deduction in the level of detail used there.

You may wonder how errors can occur in such a precise environment as a proof. One common cause is that the person writing the proof tries to prevent readers being faced with too much detail and takes a short cut using phrases like “it clearly follows that”. Another cause of invalid proofs is that a result is being used that requires certain conditions to be satisfied and they are not in fact true in the circumstances of the proof.

that increase the likelihood that a proof is correct. Those responsible for some part of a specification or some piece of code must convince all others concerned of the correctness of what has been produced. For a clean room to work effectively, it is important to suppress factors associated with power, authority, seniority, experience and personality — not easy to achieve. It is also important to remember that, in proof, the obligation is on the author to satisfy the audience, so it should be sufficient for a member of the audience to say they remain unconvinced. Nothing should leave a clean room until all have been persuaded of the correctness of what has been produced. Remember, this does not mean it is correct! Being convinced that something is true, and it actually being true, are not necessarily the same.

One of the aims of this unit is to help you to read and understand proofs, whether encountered in the literature of computer science or in a software production environment such as a clean room. Once you are comfortable reading proofs, we shall ask you to write some proofs for simple problems, similar to examples you will have met. Writing a proof is often much harder than reading a proof, but it is rewarding. You will appreciate, from what has been written above, that you should not be surprised if there are errors in your proofs. Even the most experienced amongst us continue to make such errors — indeed, there are almost certainly some errors in this and other units of the course! Even just reading and understanding proofs can be difficult. This will, however, become easier with practice, and we would encourage you to persevere if you find it hard going at first.

In this unit, we shall be concerned with a single proof technique, called **mathematical induction**. This is introduced informally in Section 1. In Section 2, we use the technique to prove results about formulae and recurrence systems, which will be of use in Unit 15. In Sections 3 and 4, we will look at a wider range of situations to which induction can be applied. For example, we shall see how the same proof technique enables us to establish results about the correctness of code involving a loop. Proof of correctness provides a stronger basis for confidence in code than does testing, since it gives an assurance that the code has the desired effect for every acceptable input.

Attempting to prove correctness can act as a useful way of detecting errors in code, such as inappropriate initialisation of a variable, or use of the wrong condition in a **while** loop. For example, suppose that the code below is intended to evaluate $INTSUM(n) = 1 + 2 + 3 + \dots + n$ (where n is an integer variable, initialised to a value greater than or equal to 1). This code will not return the correct value, and the approach described in Section 3 will help to show up such errors.

```
var nextNum in Int
var soFar in Int
soFar <-- 0
nextNum <-- n
while (NOT(nextNum == 0))
{
  nextNum <-- nextNum - 1
  soFar <-- soFar + nextNum
}
return soFar
```

Even in mathematics, there are examples of proofs that have been accepted and used for many years which have been found to be flawed at a later date.

Testing can only deal with a small finite number of sample inputs.

1 Mathematical induction

The proof technique considered in this unit is that of mathematical induction. This can be used to prove results about integers — or, more generally, results about any set whose members can in some way be labelled using integers. For example, induction can be used to prove the result that

every full binary tree of depth n has $2^n - 1$ nodes.

(A **full** binary tree is one in which every level is completely filled — so that it contains the greatest possible number of nodes for a given depth.)

See Unit 8, p. 21.

In this example, trees are labelled using their depth. To prove results about sequences, we can use the length of the sequence to associate an integer with a sequence. An important application of the technique, considered in Section 3, applies induction to code containing a loop. The number of times the loop body has been executed is used to associate an integer with the code. This enables us to use mathematical induction to prove results about the code.

Suppose that we want to prove some result for a long — possibly infinite — list of items. We can label such a list by integers. It is not possible to check the result for every individual case, because there may be infinitely many such cases. As you will see, the idea behind induction is to show two things.

- (B) The result holds for the first element.
- (I) For $k > 1$, if the result holds for the element labelled $k - 1$, then it also holds for the element labelled k .

The dominoes are arranged so that the distance between any two dominoes is less than the height of a domino.

From these two points it will then follow, by mathematical induction, that the result holds for all elements in the list. To see this, let us consider an analogy.



Figure 1.1

Suppose we have a row of dominoes, as shown in Figure 1.1, and that these are arranged so that if any one domino falls — to the left in Figure 1.1 — then it will knock down its left-hand neighbour.

We can think of the dominoes in Figure 1.1 as labelled by integers: the domino furthest to the right in the picture labelled 1, the domino to its left labelled 2, the next to the left labelled 3, and so on.

What can we say about this row of dominoes if the first domino falls?

Because of the way they are arranged, if any domino falls, then it will also make the next one fall. So if the first domino falls, then this will cause the second to fall, which in turn makes the third topple, which makes the fourth fall, which makes the fifth fall, and so on.



Figure 1.2



Figure 1.3

So from the two premises:

(B) the first domino falls;

(I) a falling domino will always topple its neighbour;

we are able to deduce the conclusion that: all the dominoes will fall.

The deduction that all dominoes fall embodies the proof technique of **mathematical induction**. We will formalise this “domino” argument in the next section, where we will also work through a number of examples of how to apply mathematical induction. Before reading on, think for a moment about what would happen if we were to start by toppling to the left the fourth, rather than the first, domino.

Let us now discuss the type of result that can be proved by induction. In Unit 13 you met the function *HANOI*, which gives the minimum number of moves needed to solve the Towers of Hanoi problem. You saw that this function can be expressed in the recursive formulation below (where n is an integer giving the number of discs).

$$\begin{aligned} HANOI(1) &= 1 & (S) \\ HANOI(n) &= 2 \times HANOI(n-1) + 1 \quad (n > 1) & (R) \end{aligned}$$

We refer to a recursive formulation such as this as a **recurrence system**. In such a recurrence system, we refer to (S) as the **initial condition**, and to (R) as the **recursive part**.

The first few values of the function *HANOI* can be calculated iteratively using the recurrence system.

$$\begin{aligned} HANOI(2) &= 2 \times HANOI(1) + 1 = 3 \\ HANOI(3) &= 2 \times HANOI(2) + 1 = 7 \\ HANOI(4) &= 2 \times HANOI(3) + 1 = 15 \\ HANOI(5) &= 2 \times HANOI(4) + 1 = 31 \end{aligned}$$

It is straightforward to use the recurrence system for evaluating *HANOI*(n) for small n , but this approach quickly becomes tedious for larger values of n . For example, if we wanted to compute *HANOI*(20) like this, we would need to use the recurrence system to calculate *HANOI*(2), *HANOI*(3), *HANOI*(4), and so on, until we got to *HANOI*(20).

Now try calculating the values given by the formula $2^n - 1$ with n equal to 1, then 2, then 3, then 4, then 5. You should get the same values as are given above for *HANOI*(n). Now, if we could prove that *HANOI*(n) is always $2^n - 1$, then it would be easier to evaluate *HANOI*(20). With the help of a calculator, $2^{20} - 1$ is readily found. It is indeed true that *HANOI*(n) always has the same value as $2^n - 1$, for $n \geq 1$, but we cannot prove this assertion by checking every possible case. There are an infinite number of integers n with $n \geq 1$, and we cannot check that *HANOI*(n) = $2^n - 1$ for each one individually. So how can we prove such a result?

We do not have to start at the integer 1 in an inductive proof.

Note that the function *HANOI* has an input n in *Int* and a precondition $n \geq 1$.

We have labelled the first condition (S), since it is also known as the **starting condition** or, in some circumstances, the **stopping condition**.

You may have guessed that this is the kind of result that we will prove using mathematical induction. Here, the elements concerned form the set of integers n with $n \geq 1$, for which we want to prove the result that $HANOI(n) = 2^n - 1$.

Many of our examples of induction in this unit will concern results for sets of integers, but do not forget that induction can also be applied to other forms of data, such as binary trees and sequences. Indeed, the result about the function $HANOI$ is really a result about the Towers of Hanoi problem itself, but we first moved to a result formulated in terms of integers — the integers used being the number of discs in the problem and the number of moves required.

2 Proofs about recurrence systems

All the proofs in this unit are based on the principle of mathematical induction. In this section we shall formalise this principle, and use it to prove results about recurrence systems.

2.1 The principle of mathematical induction

We look first at the Tower of Hanoi problem. In Section 1, we saw that the recurrence system for the function $HANOI(n)$ seemed to give the same values as the formula $2^n - 1$. In the example below, we will investigate how to prove that this is indeed the case for every integer $n \geq 1$.

EXAMPLE 2.1

Suppose that the function $HANOI(n)$ (where n is an integer and $n \geq 1$) is defined by the recurrence system below.

$$\begin{aligned} HANOI(1) &= 1 & (S) \\ HANOI(n) &= 2 \times HANOI(n-1) + 1 & (n > 1) \quad (R) \end{aligned}$$

We can show that $HANOI(n) = 2^n - 1$ for all $n \geq 1$ as follows.

Let $F(n) = 2^n - 1$. With $n = 1$, this formula gives

$$F(1) = 2^1 - 1 = 2 - 1 = 1.$$

Since $HANOI(1) = 1$ from the initial condition (S) for the recurrence system, we have the following equation.

$$HANOI(1) = F(1) \quad (B)$$

We want to extend this to show that $HANOI(n) = F(n)$ for all $n \geq 1$.

For now, suppose that we can prove the following statement for $k > 1$.

$$HANOI(k-1) = F(k-1) \Rightarrow HANOI(k) = F(k) \quad (I)$$

Our goal is to show that $F(n)$ is always the same as $HANOI(n)$.

The fact that $HANOI(1)$ is equal to $F(1)$ corresponds to the first domino falling. We have labelled this result (B) as it is the basis for the induction.

We will refer to (I) as the **inductive step**. It corresponds to the statement that a falling domino will always topple its neighbour.

We shall show later that this implication is true for all integers $k > 1$. First, let us see why this helps. We use this implication repeatedly.

$$HANOI(1) = F(1) \quad (\text{from B}).$$

$$\text{Now } HANOI(1) = F(1) \Rightarrow HANOI(2) = F(2); \text{ (I with } k = 2).$$

$$\text{Hence } HANOI(2) = F(2).$$

$$\text{Now } HANOI(2) = F(2) \Rightarrow HANOI(3) = F(3); \text{ (I with } k = 3).$$

$$\text{Hence } HANOI(3) = F(3).$$

$$\text{Now } HANOI(3) = F(3) \Rightarrow HANOI(4) = F(4); \text{ (I with } k = 4).$$

$$\text{Hence } HANOI(4) = F(4).$$

$$\vdots$$

$$\text{Now } HANOI(n-1) = F(n-1) \Rightarrow HANOI(n) = F(n); \text{ (I with } k = n).$$

$$\text{Hence } HANOI(n) = F(n).$$

All the dominoes fall over.

Since we have proved (B), the basis for the induction, the argument above shows that if we can also prove (I), the inductive step, then we can deduce that $HANOI(n) = F(n)$ for all $n \geq 1$.

Recall from Unit 9, Section 4, that an implication $p \Rightarrow q$ is **true** if we cannot have p **true** and q **false**. So to prove an implication is **true**, we suppose that its left-hand side is **true** and show that it must then follow that the right-hand side is also **true**. The left-hand side of the implication in (I) says that $HANOI(k-1) = F(k-1)$ (where $k > 1$). We suppose this is **true** and use it to show that we then have $HANOI(k) = F(k)$. We call the left-hand side of the inductive step the **induction hypothesis**, and label it (H).

The induction hypothesis is sometimes referred to as the **inductive hypothesis**.

$$HANOI(k-1) = F(k-1) = 2^{k-1} - 1 \quad (\text{H})$$

Then,

$$\begin{aligned} HANOI(k) &= 2 \times HANOI(k-1) + 1 && (\text{from the recurrence}) \\ &= 2 \times (2^{k-1} - 1) + 1 && (\text{from H}) \\ &= 2 \times 2^{k-1} - 2 + 1 && (\text{expand the bracket}) \\ &= 2^k - 1 \\ &= F(k). \end{aligned}$$

Thus, for any $k > 1$, we have that

$$HANOI(k-1) = F(k-1) \Rightarrow HANOI(k) = F(k).$$

Since we have shown (I) to be **true**, we have now completed the proof that $HANOI(n) = F(n) = 2^n - 1$ for all $n \geq 1$.

We now see how Example 2.1 can be regarded as an example of a general proof technique. In this example, the argument establishes two things:

(B) $HANOI(1) = F(1)$;

(I) for all integers $k > 1$, $HANOI(k-1) = F(k-1) \Rightarrow HANOI(k) = F(k)$.

From these two points, it follows that $HANOI(n) = F(n)$ for all $n \geq 1$. This argument follows the proof technique of mathematical induction.

Suppose that $p(n)$ is the following proposition.

$$p(n): HANOI(n) = F(n).$$

In the proof in Example 2.1 we wanted to show that $p(n)$ is **true** for all integers $n \geq 1$. The argument we gave actually establishes two things:

(B) $p(1)$ is **true**;

(I) for all integers $k > 1$, $p(k-1) \Rightarrow p(k)$ is **true**.

Now (I) is **true** for any $k > 1$, so it works for each of $k = 2, k = 3, k = 4$ and so on.

$$\begin{array}{ll}
p(1) \text{ is } \mathbf{true}; & \text{Statement (B).} \\
\text{Now } p(1) \Rightarrow & p(2) \text{ is } \mathbf{true}; \text{ (I with } k = 2). \\
\text{Hence } p(2) \text{ is } \mathbf{true}. & \\
\text{Now } p(2) \Rightarrow & p(3) \text{ is } \mathbf{true}; \text{ (I with } k = 3). \\
\text{Hence } p(3) \text{ is } \mathbf{true}. & \\
\text{Now } p(3) \Rightarrow & p(4) \text{ is } \mathbf{true}; \text{ (I with } k = 4). \\
\text{Hence } p(4) \text{ is } \mathbf{true}. & \\
\vdots & \vdots \\
\text{Now } p(n-1) \Rightarrow & p(n) \text{ is } \mathbf{true}; \text{ (I with } k = n). \\
\text{Hence } p(n) \text{ is } \mathbf{true}. &
\end{array}$$

Since (I) holds for any integer $k > 1$ we can extend this argument as far as we like. So we can deduce that $p(n)$ must be **true** for any integer $n \geq 1$.

All the dominoes fall over.

Result 2.1

Principle of mathematical induction (Version 1)

Suppose that $p(n)$ is a proposition depending on n , where n is an integer and $n > 1$.

Suppose that we can prove two things, namely:

(B) $p(1)$ is **true**;

(I) for all integers $k > 1$, $p(k - 1) \Rightarrow p(k)$ is **true**.

Then we can deduce that $p(n)$ is **true** for all integers $n \geq 1$.

(B) is often called the **starting step** or the **starting condition** as well as the **basis for induction**, and (I) is the **inductive step**. The inductive step consists of an implication as shown. To prove this implication, we suppose that $p(k-1)$ (that is, the left-hand side of the implication) holds and prove that the right-hand side must follow. In establishing the implication in (I), the assumption that $p(k-1)$ holds is known as the **induction hypothesis**, from which the truth of $p(k)$ must be deduced. Once we have established both (B) and (I), we can use mathematical induction to deduce that $p(n)$ holds for all $n > 1$.

We shall normally refer to (B) as the **starting step**.

At first sight, the principle of mathematical induction may seem to be of rather restricted application, since it looks as if it can only be used to prove results about integers. However, as mentioned in the introduction, induction can be applied more generally. For example, it can be applied to trees, by taking the integer n to be the depth of a tree, and formulating the proposition $p(n)$ in a suitable way. Indeed, induction can be used to prove a wide variety of results. In this section we concentrate on recurrence systems.

Some examples are given in Sections 3 and 4.

Version 1 of the principle of mathematical induction contains a couple of restrictions that are not necessary. Firstly, we do not have to start at 1. We might want to start at 0, or 2, or indeed at any integer. This is feasible, and the corresponding version of the principle of mathematical induction is given below (where M is used for the general integer at which the process is started).

Result 2.2

Principle of mathematical induction (Version 2)

Suppose that $p(n)$ is a proposition depending on n , where n is an integer and $n \geq M$ (where M might be any integer).

Suppose that we can prove two things, namely:

(B) $p(M)$ is **true**;

(I) for all integers $k > M$, $p(k-1) \Rightarrow p(k)$ is **true**.

Then we can deduce that $p(n)$ is **true** for all integers $n \geq M$.

2.2 Proving formulae for recurrence systems

In Section 2.1, we saw how to prove that the function $HANOI(n)$, defined through a recurrence system, gives the same values as the formula $2^n - 1$. We say that $2^n - 1$ is a **solution** of the recurrence system defining $HANOI$. We now look at some more examples of the same type, where we prove that a given formula is a solution of a recurrence system.

Activity 2.1

Two functions $INTSUM$ and $FORM$ both have an input n that is an integer, with $n \geq 1$. The semantics of these functions are given below. Note that $FORM(n)$ is given by a formula, while $INTSUM(n)$ is given by a recurrence system.

$$\begin{aligned} INTSUM(1) &= 1 & (S) \\ INTSUM(n) &= INTSUM(n-1) + n \quad (n > 1) & (R) \end{aligned}$$

$$FORM(n) = \frac{1}{2}n(n+1)$$

Complete the table below by calculating the values of each of these two functions for $n = 1, n = 2, n = 3$ and $n = 4$.

n	$INTSUM(n)$	$FORM(n)$
1		
2		
3		
4		

The information given in the postcondition in a function specification forms the **semantics** of the function.

Remember that $\frac{1}{2}n(n+1)$ means the same as $\frac{1}{2} \times n \times (n+1)$.

Discussion Using the formula with $n = 1$, then $n = 2$, then $n = 3$, then $n = 4$, we get: $FORM(1) = \frac{1}{2} \times 1 \times 2 = 1$; $FORM(2) = \frac{1}{2} \times 2 \times 3 = 3$; $FORM(3) = \frac{1}{2} \times 3 \times 4 = 6$; $FORM(4) = \frac{1}{2} \times 4 \times 5 = 10$.

Now look at the recurrence system. This gives $INTSUM(1) = 1$. Using the recursive part of the recurrence (Equation (R) above), with $n = 2$, then $n = 3$, then $n = 4$, we get:

$$\begin{aligned} INTSUM(2) &= INTSUM(1) + 2 = 1 + 2 = 3; \\ INTSUM(3) &= INTSUM(2) + 3 = 3 + 3 = 6; \\ INTSUM(4) &= INTSUM(3) + 4 = 6 + 4 = 10. \end{aligned}$$

The completed table of values is:

n	$INTSUM(n)$	$FORM(n)$
1	1	1
2	3	3
3	6	6
4	10	10

In Activity 2.1, we saw that the functions $INTSUM(n)$ and $FORM(n)$ give the same values for n equal to 1, 2, 3 and 4. But do these functions give the same values for all integers $n \geq 1$? This is indeed so. To prove that the formula $\frac{1}{2}n(n+1)$ is a solution of the recurrence system for $INTSUM(n)$, we can use an approach similar to that used to show that $HANOI(n) = 2^n - 1$ for any integer $n \geq 1$. In the next activity, we will prove the two steps needed for mathematical induction.

Activity 2.2

Suppose that the function $INTSUM(n)$ (where n is an integer and $n \geq 1$) is defined by the following recurrence system.

$$\begin{aligned} INTSUM(1) &= 1 \\ INTSUM(n) &= INTSUM(n-1) + n \quad (n > 1) \end{aligned}$$

Prove that $INTSUM(n) = \frac{n(n+1)}{2}$ for all $n \geq 1$.

$\frac{n(n+1)}{2}$ is another way of writing $\frac{1}{2}n(n+1)$. We may also write this expression as $n(n+1)/2$.

Discussion As in Activity 2.1, let $FORM(n) = \frac{n(n+1)}{2}$.

We saw there that $INTSUM(1) = FORM(1)$, which establishes the starting step.

Now we need to show the inductive step, that, for $k > 1$:

$$INTSUM(k-1) = FORM(k-1) \Rightarrow INTSUM(k) = FORM(k).$$

So, for $k > 1$, suppose that the formula $FORM(k-1)$ gives $INTSUM(k-1)$ correctly (that is, the left-hand side of the implication holds). This is the induction hypothesis, and gives us the equation below.

$$INTSUM(k-1) = FORM(k-1) = \frac{(k-1)(k-1+1)}{2} = \frac{(k-1)k}{2} \quad (\text{H})$$

From the recurrence and the induction hypothesis, we obtain:

$$\begin{aligned} INTSUM(k) &= INTSUM(k-1) + k && \text{(from the recurrence)} \\ &= \frac{(k-1)k}{2} + k && \text{(from H, the induction hypothesis)} \\ &= \frac{(k-1)k + 2k}{2} && \text{(putting over a common denominator)} \\ &= \frac{k(k-1+2)}{2} && \text{(taking out a common factor } k) \\ &= \frac{k(k+1)}{2}. \end{aligned}$$

Now notice that the final expression on the right above is $FORM(k)$.

Thus, for any integer $k > 1$, if the formula $FORM$ gives $INTSUM(k - 1)$ correctly, then it also gives $INTSUM(k)$ correctly, establishing the inductive step.

Having shown both that the starting step holds and that the inductive step holds, we can now deduce, using the principle of mathematical induction, that, for all integers $n \geq 1$, the formula $FORM(n) = \frac{1}{2}n(n + 1)$ gives the same values as the function $INTSUM(n)$ defined by the recurrence system.

In Example 2.1 and Activity 2.2 we discussed how to prove that a formula is a solution to a recurrence system. Expressed informally, the method of proof was to show the following.

- (B) The formula produces the correct result for the initial condition.
- (I) If the formula gives the $k - 1$ term of the recurrence correctly, then it also gives the k term correctly.

We then used mathematical induction to deduce that the formula is a solution to the recurrence system.

We now look at a result about binary trees. The first step is to formulate the result in terms of a recurrence system, where n is the depth of the tree. In Figure 2.1, note the number of leaves in each case: (a) 1; (b) 2; (c) 4.

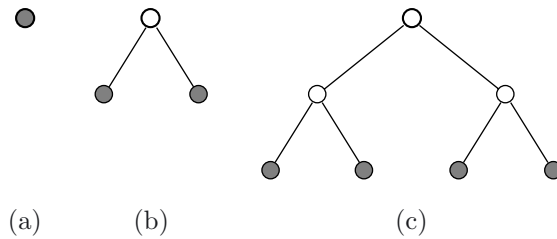


Figure 2.1 Full binary trees of depth: (a) 1, (b) 2 and (c) 3. The leaf nodes are shaded.

Activity 2.3

Let $LEAVES(n)$ be the number of leaves in a full binary tree of depth n .

- (a) Explain why we have the following recurrence system.

$$LEAVES(1) = 1 \quad (S)$$

$$LEAVES(n) = 2 \times LEAVES(n - 1) \quad (n > 1) \quad (R)$$

- (b) Hence use mathematical induction to prove that, for $n \geq 1$:

$$LEAVES(n) = 2^{n-1}.$$

Discussion

- (a) A full binary tree of depth 1 consists of just the root node, and this is a leaf in this case (see Figure 2.1(a)). So $LEAVES(1) = 1$.

In a full binary tree of depth n (with $n > 1$), the number of nodes at level $n - 1$ is equal to the number of leaves in a full binary tree of depth $n - 1$, and so is $LEAVES(n - 1)$. Now in the full binary tree of depth n , each of these nodes has two children, and each of these is a leaf. That gives all the leaves, so we have, for $n > 1$:

$$LEAVES(n) = 2 \times LEAVES(n - 1).$$

So $LEAVES$ must satisfy the given recurrence system.

- (b) Let $F(n) = 2^{n-1}$. Putting $n = 1$ into this formula gives $F(1) = 2^0 = 1$. This is the same value as $LEAVES(1)$ from the recurrence, so

$$LEAVES(1) = F(1),$$

and this establishes the starting step.

Now we need to show the inductive step, i.e. for $k > 1$:

$$LEAVES(k-1) = F(k-1) \Rightarrow LEAVES(k) = F(k).$$

So, for $k > 1$, suppose that the formula $F(k-1)$ gives $LEAVES(k-1)$ correctly (that is, the left-hand side of the implication holds). This is the induction hypothesis, and gives us the equation below.

$$LEAVES(k-1) = F(k-1) = 2^{k-2} \quad (\text{H})$$

From the recurrence and the induction hypothesis, we obtain:

$$\begin{aligned} LEAVES(k) &= 2 \times LEAVES(k-1) && (\text{from the recurrence}) \\ &= 2 \times 2^{k-2} && (\text{from H}) \\ &= 2^{k-1} \\ &= F(k). \end{aligned}$$

So for any integer $k > 1$, if the formula gives $LEAVES(k-1)$ correctly, then it also gives $LEAVES(k)$ correctly, establishing the inductive step.

Having shown that both the starting step and the inductive step hold, we can now deduce, using the principle of mathematical induction, that, for all integers $n \geq 1$, the formula $F(n) = 2^{n-1}$ gives the same values as the function $LEAVES(n)$ defined by the recurrence system.

Activity 2.4

Consider the function *POWERTWO* given by $POWERTWO(n) = 2^n$, for $n \geq 0$. How does a recurrence system for *POWERTWO* differ from that given for *LEAVES* in Activity 2.3?

Discussion The function *POWERTWO* satisfies, for $n > 0$, the equation

$$POWERTWO(n) = 2 \times POWERTWO(n-1).$$

This is the same recursive condition as is satisfied by *LEAVES*. Since the formula for *POWERTWO* is different from that for *LEAVES*, there must be some difference in the recurrence systems they satisfy! We need also to check the initial condition. Note that the smallest n for which *POWERTWO* is defined is 0. Since $2^0 = 1$, the initial condition for *POWERTWO* is

$$POWERTWO(0) = 1,$$

which you will see is different from the initial condition for *LEAVES*. Whenever you produce a recursive formulation of the semantics of a function, it is a sensible precaution to check that you have the correct initial condition.

A recurrence system need not have its initial condition at 1. Having an initial condition at some other integer makes little difference to the method for proving that a given formula is a solution of the recurrence. We just need to check that the formula gives the correct value for the initial condition, and then continue as before.

This approach depends on Version 2 of the principle of mathematical induction — see page 11.

Strategy 2.1

To prove that a formula is a solution of a recurrence system

Suppose $U(n)$ is defined in terms of $U(n - 1)$ by a recurrence system with an initial condition at $n = M$, that $F(n)$ is given as a formula, and we wish to prove that $U(n) = F(n)$ for all integers $n \geq M$.

1. Show that the formula gives the same value as the initial condition for the recurrence; that is, show that $U(M) = F(M)$.
2. Show that, for all $k > M$: if the formula gives $U(k - 1)$ correctly, then the formula also gives $U(k)$ correctly.

For step 2, substitute $k - 1$ for n in the formula, to find an expression for $F(k - 1)$, then replace $U(k - 1)$ by $F(k - 1)$ in the recursive part of the recurrence system.

The following example illustrates how an inductive proof of this form can be expressed in a fairly succinct way.

EXAMPLE 2.2

For n an integer with $n \geq 0$, let $U(n)$ be defined by the recurrence system below.

$$\begin{aligned} U(0) &= 2 & (S) \\ U(n) &= 3 \times U(n - 1) - 2 \quad (n > 0) & (R) \end{aligned}$$

A proof that $U(n) = 3^n + 1$ for all integers $n \geq 0$ can be presented as follows.

Let $F(n) = 3^n + 1$. With $n = 0$, we have $F(0) = 3^0 + 1 = 1 + 1 = 2$. This is the same value as in the initial condition, $U(0) = 2$, for the recurrence system.

Now suppose, for $k > 0$, that the formula gives $U(k - 1)$ correctly, so that we have the following equation.

$$U(k - 1) = F(k - 1) = 3^{k-1} + 1 \quad (H)$$

Then

$$\begin{aligned} U(k) &= 3 \times U(k - 1) - 2 \quad (\text{from the recurrence}) \\ &= 3 \times (3^{k-1} + 1) - 2 \quad (\text{from H}) \\ &= 3 \times 3^{k-1} + 3 - 2 \\ &= 3^k + 1 \\ &= F(k). \end{aligned}$$

Thus, for all $k > 0$, if the formula F gives $U(k - 1)$ correctly then it also gives $U(k)$ correctly, so establishing the inductive step.

Having shown that the starting condition and the inductive step hold, we can deduce, using the principle of mathematical induction, that the formula $F(n) = 3^n + 1$ gives the same values for all $n \geq 0$ as the function $U(n)$ defined by the recurrence.

To explain how we used mathematical induction in the argument in Example 2.2, let $p(n)$ be the following proposition.

$$p(n): U(n) = F(n)$$

We showed that $U(0) = F(0)$, i.e. $p(0)$ is **true**.

We then showed the implication

$$U(k-1) = F(k-1) \Rightarrow U(k) = F(k), \text{ for all } k > 0.$$

So, in terms of the proposition p , we showed that, for all $k > 0$:

$$p(k-1) \Rightarrow p(k) \text{ is } \mathbf{true}.$$

Then, by the principle of mathematical induction (Version 2 with $M = 0$), we can deduce that $p(n)$ is **true** for all $n \geq 0$. This is the result we wanted: that $U(n) = F(n)$ holds for all $n \geq 0$.

2.3 Formulae for sums

Activity 2.5

Let n be an integer with $n \geq 0$, let $F(n) = 2^{n+1} - 1$, and let $SUMPOWERTWO(n)$ be defined by the recurrence system below.

$$SUMPOWERTWO(0) = 1 \quad (\text{S})$$

$$SUMPOWERTWO(n) = SUMPOWERTWO(n-1) + 2^n \quad (n > 0) \quad (\text{R})$$

Prove that $SUMPOWERTWO(n) = F(n)$ for all integers $n \geq 0$.

Discussion With $n = 0$, the formula $F(n) = 2^{n+1} - 1$ gives

$$F(0) = 2^{0+1} - 1 = 2^1 - 1 = 2 - 1 = 1.$$

This is the same as the initial condition, (S), for the recurrence system.

Now suppose, for $k > 0$, that the formula $F(k-1)$ gives $SUMPOWERTWO(k-1)$ correctly, so that we have the following equation.

$$SUMPOWERTWO(k-1) = F(k-1) = 2^{k-1+1} - 1 = 2^k - 1 \quad (\text{H})$$

Then,

$$\begin{aligned} SUMPOWERTWO(k) &= SUMPOWERTWO(k-1) + 2^k \quad (\text{from the recurrence}) \\ &= 2^k - 1 + 2^k \quad (\text{from H}) \\ &= 2 \times 2^k - 1 \\ &= 2^{k+1} - 1 \\ &= F(k). \end{aligned}$$

Thus, for all $k > 0$, if the formula F gives $SUMPOWERTWO(k-1)$ correctly then it also gives $SUMPOWERTWO(k)$ correctly, so establishing the inductive step.

Having shown that the starting condition and the inductive step hold, we can deduce, using the principle of mathematical induction, that the formula $F(n) = 2^{n+1} - 1$ gives the same values for all $n \geq 0$ as the function $SUMPOWERTWO(n)$ defined by the recurrence.

The recurrence system in Activity 2.5 is a recursive formulation of a function that expresses a sum, where the terms being summed are of the form 2^i . The result in the activity can be expressed as the following sum.

$$1 + 2 + 2^2 + 2^3 + \dots + 2^n = 2^{n+1} - 1$$

Since $1 = 2^0$ and $2 = 2^1$, the first two terms in this sum conform to the same general pattern as the later terms.

Below, we give some results about sums. We established the first result in Activity 2.2. The second is covered in Exercise 2.3. The third is a generalised version of the sum of powers of 2 that was covered in Activity 2.5. (The formula given below works not just for powers of 2, but for any real number r except 1.)

Result 2.3

$$1 + 2 + 3 + \dots + n = \frac{1}{2}n(n + 1).$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{1}{6}n(n + 1)(2n + 1).$$

$$1 + r^1 + r^2 + r^3 + \dots + r^n = \frac{r^{n+1} - 1}{r - 1}, \text{ for any real number } r \neq 1.$$

$r^0 = 1$ for any real number r .

Aside

You may have seen such sums written using the following notation, called the **sigma notation** since Σ is the Greek letter capital sigma.

$1 + 2 + 3 + \dots + n$ is written as $\sum_{i=1}^{i=n} i$.

In a similar way, $2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^n$ is written as $\sum_{i=0}^{i=n} 2^i$.

Here, 2^i is the general term in the sum, and the $i = 0$ and $i = n$ indicate that we add up the values taken by 2^i for i equal to 0, 1, 2, 3 and so on, up to i equal to n .

You may see this notation used in other texts, but we shall not need it in this course.

End of Aside

2.4 Extending the principle of induction

For completeness, we end the section by mentioning a way in which the principle of mathematical induction can be extended. In Example 2.3 overleaf, $U(n)$ is defined by the following recurrence system.

$$\begin{aligned} U(0) &= 0 \\ U(1) &= 1 \\ U(n) &= 2U(n-1) - U(n-2) + 2 \quad (n > 1) \end{aligned}$$

Here, the recursive expression for $U(n)$ depends upon both $U(n-1)$ and $U(n-2)$. In such a case, we need to modify the method for showing that $U(n) = F(n)$. We suppose that the formula F correctly gives both $U(k-1)$ and $U(k-2)$ in the expression

$$U(k) = 2U(k-1) - U(k-2) + 2,$$

and try to deduce that the formula must then give $U(k)$ correctly. In such a case the first two terms of U are given explicitly, not just the first term. From n equal to 2 onwards, the recursive part of the recurrence system is used to define $U(n)$.

This generalised method draws on a version of the principle of mathematical induction that is given below. This uses the idea that, when we seek to deduce that $p(k)$ is **true**, we already know that all the propositions $p(0), p(1), p(2), \dots$ and so on, up to $p(k-1)$ are **true**.

Read Only

The material in this subsection will not be assessed.

Result 2.4

Principle of mathematical induction (Version 3)

Suppose that $p(n)$ is a proposition depending on n , where n is an integer and $n \geq 0$. Suppose that we can prove three things, namely:

1. $p(0)$ is **true**;
2. $p(1)$ is **true**;
3. for all integers $k > 1$, if $p(j)$ is **true** for all integers j with $0 \leq j \leq k - 1$, then $p(k)$ is **true**; or, expressed as an implication:

$$(p(0) \wedge p(1) \wedge \dots \wedge p(k-1)) \Rightarrow p(k) \text{ is } \mathbf{true}.$$

Then we can deduce that $p(n)$ is **true** for all integers $n \geq 0$.

This does not express the principle of mathematical induction in the most general form possible. For one thing, we do not have to start at 0. Also, there could be more than two individual cases to start the process. Expressing the principle in the most general form possible is awkward, and is omitted. The key requirements are to have

- one or more starting cases, and
- an inductive step

constructed in a way that leaves no gaps.

EXAMPLE 2.3

For n an integer, with $n \geq 0$, let $F(n) = n^2$, and let $U(n)$ be defined by the recurrence system below.

$$\begin{aligned} U(0) &= 0 \\ U(1) &= 1 \\ U(n) &= 2U(n-1) - U(n-2) + 2 \quad (n > 1) \end{aligned}$$

To prove that $U(n) = F(n)$ for all integers $n \geq 0$, we argue as follows.

Putting $n = 0$ in the formula gives $F(0) = 0^2 = 0$, which is the same value as is given for $U(0)$ in the recurrence.

Putting $n = 1$ in the formula gives $F(1) = 1^2 = 1$, which is the same value as is given for $U(1)$ in the recurrence.

For $k > 1$, $U(k)$ is given by the recursive part of the recurrence system. We want to show that $U(k) = F(k)$, for $k > 1$. Version 3 of the principle of mathematical induction allows us to suppose that $U(j) = F(j)$ for all j between 0 and $k - 1$. Because $U(k)$ is defined only in terms of $U(k - 1)$ and $U(k - 2)$, we need only suppose that $U(k - 1)$ and $U(k - 2)$ are correctly given by the formula F , so that:

$$\begin{aligned} U(k-1) &= F(k-1) = (k-1)^2 \\ U(k-2) &= F(k-2) = (k-2)^2. \end{aligned}$$

Then, for $k > 1$,

$$\begin{aligned} U(k) &= 2U(k-1) - U(k-2) + 2 && \text{(from the recurrence)} \\ &= 2(k-1)^2 - (k-2)^2 + 2 && \text{(induction hypothesis)} \\ &= 2(k^2 - 2k + 1) - (k^2 - 4k + 4) + 2 \\ &= 2k^2 - 4k + 2 - k^2 + 4k - 4 + 2 \\ &= k^2 \\ &= F(k). \end{aligned}$$

In this version of mathematical induction, both these equations form the induction hypothesis.

Using mathematical induction, we can then deduce that the formula $F(n)$ gives the same values as the recurrence system gives for $U(n)$, for all integers $n \geq 0$.

To relate this to the principle of mathematical induction (Version 3), note that we have shown that:

$$U(n) = F(n) \text{ for } n = 0;$$

$$U(n) = F(n) \text{ for } n = 1;$$

for all $k > 1$, if $U(j) = F(j)$ for all j with $0 \leq j \leq k - 1$ then $U(k) = F(k)$.

Objectives for Section 2

After studying this section you should be able to do the following.

- Recognise and use the terminology: recurrence system, initial condition and recursive part of a recurrence system, solution of a recurrence system, mathematical induction, starting condition for induction, basis for induction, inductive step, induction hypothesis.
- Use the principle of mathematical induction to prove that a given formula is a solution of a recurrence system — for a system similar to those you have met in this section.

Exercises on Section 2

Exercise 2.1

Let $U(n)$ be defined by the recurrence system below.

$$U(2) = 4 \quad (\text{S})$$

$$U(n) = U(n - 1) + 5 \quad (n > 2) \quad (\text{R})$$

Prove that the formula $F(n) = 5n - 6$ gives the same values as the function $U(n)$ for all integers n with $n \geq 2$.

Exercise 2.2

Let $T(n)$ be defined by the recurrence system below.

$$T(0) = 5 \quad (\text{S})$$

$$T(n) = 2T(n - 1) + 3 \quad (n > 0) \quad (\text{R})$$

Prove that the formula $F(n) = 8 \times 2^n - 3$ gives the same values as the function $T(n)$ for all integers n with $n \geq 0$.

Exercise 2.3

If n is an integer with $n \geq 1$, let $SUMSQ(n)$ be the sum of the squares of each integer from 1 to n , i.e.

$$SUMSQ(n) = 1^2 + 2^2 + 3^2 + \dots + n^2.$$

(a) Give a recurrence system satisfied by $SUMSQ(n)$.

(b) Show that, for all integers $n \geq 1$, $SUMSQ(n) = \frac{1}{6}n(n + 1)(2n + 1)$.

Exercise 2.4

Let $NODES(n)$ be the number of nodes in a full binary tree of depth n .

- (a) In Activity 2.3, we found a formula for $LEAVES(n)$, the number of leaves in a full binary tree. Using that, explain why $NODES(n)$ must satisfy the recurrence below.

$$\begin{aligned} NODES(1) &= 1 & (S) \\ NODES(n) &= NODES(n-1) + 2^{n-1} \quad (n > 1) & (R) \end{aligned}$$

Use the formula for sums of powers to deduce a formula for $NODES(n)$.

- (b) The number of nodes in any tree is the sum of the number of nodes in the left subtree and the number of nodes in the right subtree plus 1 (for the root node). Use this idea to find a different recurrence system that $NODES(n)$ must satisfy. Show that the formula for $NODES(n)$ that you obtained in (a) is also a solution for this alternative recurrence system for $NODES(n)$.

3 Proofs about algorithms involving while loops

In Block 1 we used testing to check whether an implementation satisfied its specification for a sample of input values. Testing can uncover errors in code, but it may not always do so because of the limited number of cases we can check. Mostly it is possible to test only a small proportion of all the valid inputs, and errors may occur for input values not tested. For safety critical systems we would want more reassurance that the code is correct than can be provided by some limited amount of testing. Even when testing does show up errors in an implementation, it may not be clear where these errors lie. Mathematical induction can be applied to code involving loops to prove its correctness or to locate errors such as in the initialisation of variables or in the formulation of loop conditions.

In this section we will look at **while** loops and use mathematical induction to establish their correctness. To this end we consider loop invariants (quantities that remain unchanged by one execution of the loop body). We will also show how we can use the same method to find errors in an implementation.

3.1 A loop invariant condition

This subsection provides an introduction to the approach we will use for proving results about code involving a **while** loop. You may find some of the ideas a little difficult when you first meet them. Avoid getting bogged down while studying this subsection: aim to get an overview of the approach without necessarily worrying if there are some details you do not follow at a first reading.

A function *SUMSTACK* is specified below.

function *SUMSTACK*(*s* in **Stack of Int**) **return** in **Int**

pre **true**.

post The returned value is the sum of the numbers in *s*. If *s* is empty, the returned value is 0.

The data type **Stack** is convenient for illustrating the ideas here. It was introduced in Unit 3.

Activity 3.1

Explain, by reference to the postcondition of the function *SUMSTACK*, why the following must be true for any non-empty stack, s , of integers.

$$SUMSTACK(s) = PEEK(s) + SUMSTACK(POP(s))$$

Discussion $SUMSTACK(s)$ is the sum of the numbers in the stack s .

$SUMSTACK(POP(s))$ is the sum of the numbers in the stack $POP(s)$.

Now $POP(s)$ is the stack formed by removing the top item from s , and this top item is $PEEK(s)$. So $PEEK(s) + SUMSTACK(POP(s))$ gives the sum of all the numbers in s , and so is the same as $SUMSTACK(s)$. So we have

See Unit 13, Activity 2.3.

$$PEEK(s) + SUMSTACK(POP(s)) = SUMSTACK(s).$$

Code intended to implement the function *SUMSTACK* is given below. Since we are concerned here with proving that code provides a correct implementation of a given specification, we need to maintain a clear distinction between the implementation and the specification. Any reference to “the function *SUMSTACK*” is a reference to the specification. We have given the implementation its own name, *impSumStack*, which we will use when we want to refer to the implementation.

```
function SUMSTACK(s)
{
    // impSumStack
    var t in Stack of Int
    var soFar in Int
    t <-- s
    soFar <-- 0
    while (NOT(SIZE(t) == 0))
    {
        soFar <-- soFar + PEEK(t)
        t <-- POP(t)
    }
    return soFar
}
```

Activity 3.2

Trace the states of the variables as *impSumStack* is executed when the input stack s is $[5, 7, 12, 3, 5]$. Hence determine the value returned for this input. Is the value you obtain equal to $SUMSTACK(s)$?

Discussion A trace of the variables t and $soFar$ is given below.

Comment	$soFar$	t	$NOT(SIZE(t) == 0)$
Before loop	0	[5, 7, 12, 3, 5]	true
Enter loop	5	[5, 7, 12, 3]	true
	8	[5, 7, 12]	true
	20	[5, 7]	true
	27	[5]	true
Loop end	32	[]	false

The returned value is 32.

Referring to the postcondition of $SUMSTACK$, we see that the value returned as $SUMSTACK([5, 7, 12, 3, 5])$ should be $5 + 7 + 12 + 3 + 5 = 32$. So the value returned by $impSumStack$ for this input does satisfy the postcondition in the specification of $SUMSTACK$.

As illustrated in Activity 3.2, testing can verify that the correct value is returned for a particular input. Here, though, we are looking for a form of argument that will apply to any valid input.

The idea underlying $impSumStack$ is as follows. The variable $soFar$ gives the sum of the numbers in the original stack s that have been removed from s to leave t , while t gives the numbers from s remaining to be added. Now if our code expresses this design idea correctly, then the following will be true:

$soFar + (\text{the sum of the numbers remaining in } t) = \text{the sum of the numbers in } s$.

This condition should hold after each execution of the body of the loop. We can express it more formally as

$soFar + SUMSTACK(t) = SUMSTACK(s)$.

It is important when you interpret this equation to be clear that $SUMSTACK$ means the function $SUMSTACK$ as given in the specification. $SUMSTACK(s)$ and $SUMSTACK(t)$ in the equation above are not references to values returned by the code given to implement $SUMSTACK$.

Activity 3.3

Calculate the quantity $soFar + SUMSTACK(t)$ after each execution of the body of the loop in $impSumStack$ when the input stack s is [5, 7, 12, 3, 5].

As traced in Activity 3.2.

Discussion We get the following values.

Comment	$soFar$	t	$soFar + SUMSTACK(t)$
Before loop	0	[5, 7, 12, 3, 5]	$0 + 32 = 32$
Enter loop	5	[5, 7, 12, 3]	$5 + 27 = 32$
	8	[5, 7, 12]	$8 + 24 = 32$
	20	[5, 7]	$20 + 12 = 32$
	27	[5]	$27 + 5 = 32$
Loop end	32	[]	$32 + 0 = 32$

The purpose of the argument that we are about to give is to establish that the value returned by $impSumStack$ must satisfy the specification of $SUMSTACK$. So while we go through the argument, we must be careful to maintain a distinction between the following two things.

- The value $SUMSTACK(s)$ as determined by reference to the postcondition in the specification.
- The value that will be returned by the code in the proposed implementation $impSumStack$ when it is supplied with s as an input.

We want these two things to be the same! But they are potentially different, and we must think of them as separate things until they are proved to be the same. A **proof of correctness** will establish that the value returned by the implementation must always satisfy the postcondition in the specification — for any input value(s) of the types given in the signature that satisfy the precondition.

As noted above, the key idea in the design of this implementation is that

$$soFar + SUMSTACK(t) = SUMSTACK(s) \quad (\text{LIC})$$

You will see below why we have called this equation (LIC).

should be true after each execution of the body of the loop in $impSumStack$. To prove this, we can use mathematical induction. Before we look at how to do that, let us see how this information can be used to prove that the implementation is correct.

For now, suppose we have established that Equation (LIC) holds. The key use of this equation will be that this condition holds when execution of the loop is finished. Denote by t_F and $soFar_F$ the values of the variables t and $soFar$ just after the loop is left. Then, from Equation (LIC), we know that:

$$soFar_F + SUMSTACK(t_F) = SUMSTACK(s). \quad (1)$$

Now we know that a **while** loop stops executing when the **while** condition becomes **false**. In $impSumStack$, the condition is $NOT(SIZE(t) == 0)$. This will become **false** when $SIZE(t) == 0$ becomes **true**. So the **while** loop will stop execution when $SIZE(t)$ is 0 (that is, when t becomes empty). Thus we have:

$$SIZE(t_F) \text{ is } 0. \quad (2)$$

To complete the proof that the implementation is correct, we combine the two pieces of information given in Equations (1) and (2). Since an empty stack contains no integers to add up, the postcondition of the specification of $SUMSTACK$ states that $SUMSTACK$ of an empty stack has value 0. Since $SIZE(t_F)$ is 0, we know that $SUMSTACK(t_F) = 0$. Substituting this into Equation (1) we now can deduce that:

$$soFar_F = SUMSTACK(s). \quad (3)$$

Equation (3) tells us that: the value of the variable $soFar$ just after the **while** loop stops is equal to the value returned by the function $SUMSTACK$ with input s as given by the postcondition in the specification. Now $soFar_F$ is the value returned by the code in the implementation. So establishing that Equation (3) holds will prove that the value returned by the implementation $impSumStack$ must be equal to the value $SUMSTACK(s)$ as given by the postcondition in the specification. Thus by establishing Equation (3) we prove that $impSumStack$ is a correct implementation of $SUMSTACK$.

There is some terminology that is often used in the context of such proofs. The quantity $soFar + SUMSTACK(t)$ is unchanged by a single execution of the loop body in $impSumStack$. Such a quantity is called a **loop invariant**. The condition

Of course, we have not proved this yet!

$$soFar + SUMSTACK(t) = SUMSTACK(s) \quad (\text{LIC})$$

that holds after each execution of the loop body is called a **loop invariant condition**.

So far, we have only discussed the strategy for using a loop invariant in a proof of correctness. We have not proved that we actually have a loop invariant and so, strictly, are not yet justified in talking about a loop invariant. We should not really call a quantity an invariant until we have proved that it is one! We now turn to the details of how we can establish that we have a loop invariant. The proof will use induction.

Let n be the number of times that the body of the loop in *impSumStack* has been executed. To use induction to prove that (LIC) holds for all $n \geq 0$ we need to show two things.

- (B) $soFar + SUMSTACK(t) = SUMSTACK(s)$ is true before the first execution of the loop body.
- (I) For $k > 0$, if $soFar + SUMSTACK(t) = SUMSTACK(s)$ holds after $k - 1$ executions of the loop body, then it still holds after k executions.

Let us see how we can establish these points for *impSumStack*.

(B) When the loop is first entered, *soFar* will have the value 0, from the assignment $soFar \leftarrow 0$, and t will have the value s , from the assignment $t \leftarrow s$. So, before the loop, the quantity $soFar + SUMSTACK(t)$ will have the value

$$0 + SUMSTACK(s).$$

This is certainly equal to $SUMSTACK(s)$, as required for (LIC).

(I) Now suppose that after some number of executions of the loop body, $soFar + SUMSTACK(t) = SUMSTACK(s)$ is true. What is the situation after the loop body is executed once more? To express this, it is convenient to introduce a notation for the state of each of the variables after an arbitrary number of executions of the loop body. We will write $soFar_n$ for the state of *soFar* after the loop body has been executed n times, and t_n for the state of t after the loop body has been executed n times. We now consider a single execution of the loop body, the k th execution. The k th execution of the statements in the body of the loop changes the states of the variables from t_{k-1} and $soFar_{k-1}$ to t_k and $soFar_k$. Looking at the statements in the loop body, we must have the following equations.

$$\begin{aligned} soFar_k &= soFar_{k-1} + PEEK(t_{k-1}) \\ t_k &= POP(t_{k-1}) \end{aligned}$$

The next step in the proof is to use these equations to see how a single execution of the loop body affects the proposed loop invariant. Using the two equations above, we have

$$soFar_k + SUMSTACK(t_k) = soFar_{k-1} + PEEK(t_{k-1}) + SUMSTACK(POP(t_{k-1})).$$

Now, using the result of Activity 3.1 (with $s = t_{k-1}$), we have that

$$PEEK(t_{k-1}) + SUMSTACK(POP(t_{k-1})) = SUMSTACK(t_{k-1}).$$

So we have shown that

$$soFar_k + SUMSTACK(t_k) = soFar_{k-1} + SUMSTACK(t_{k-1}).$$

So, if $soFar_{k-1} + SUMSTACK(t_{k-1}) = SUMSTACK(s)$ is true, then $soFar_k + SUMSTACK(t_k) = SUMSTACK(s)$ must also be true.

We have now established the two points noted above, namely that:

- (B) $soFar + SUMSTACK(t) = SUMSTACK(s)$ holds before the loop is entered, and
- (I) if $soFar + SUMSTACK(t) = SUMSTACK(s)$ holds after $k - 1$ executions of the loop body, then it also holds after k executions.

An invariant is a quantity that is unchanged by some action — in this case the execution of the loop body.

At this point, there have been no executions of the loop body, so $n = 0$.

This step checks that the code gives suitable initial values to the variables *soFar* and t .

This is a key step in the proof. It must refer to the code statements in the body of the loop.

We are now justified in calling $soFar + SUMSTACK(t)$ a loop invariant. We have shown that a single execution of the loop body does not change this quantity.

We can then deduce, using mathematical induction, that the condition $soFar + SUMSTACK(t) = SUMSTACK(s)$ holds after an arbitrary number of executions of the loop body. We have proved Equation (LIC) is a loop invariant condition for *impSumStack*. We saw earlier how we can then use this information to complete a proof of correctness for *impSumStack*.

We can relate this form of proof to the principle of mathematical induction in the following way. The proposition $p(n)$ we are concerned with is this:

$soFar + SUMSTACK(t) = SUMSTACK(s)$ holds after n executions of the body of the loop.

(B) establishes that $p(n)$ is **true** with $n = 0$.

(I) establishes, for $k > 0$, that: $p(k - 1) \Rightarrow p(k)$ is **true**.

So together they will establish, using mathematical induction, that $p(n)$ is **true** for all integers $n \geq 0$.

In more practical terms,

(B) makes sure that the variables have been given the correct initial values prior to the loop;

(I) helps to ensure that the code in the body of the loop is correct.

The diagrams in Figure 3.1 illustrate a single execution of the loop body in *impSumStack*, and its effect on the variables and on the loop invariant. In the situation illustrated in Figure 3.1, we have

$$SUMSTACK(t_k) + soFar_k = SUMSTACK(t_{k-1}) + soFar_{k-1}$$

because: $soFar_k$ is formed by adding 4 (the item at the top of t_{k-1}) to $soFar_{k-1}$, so that $soFar_k = soFar_{k-1} + 4$, while t_k is formed by *POP*ping the item 4 from the top of the stack t_{k-1} , so that $SUMSTACK(t_k) = SUMSTACK(t_{k-1}) - 4$.

$$\begin{array}{lll} \text{Just before loop (0 executions)} & soFar_0 = 0 & [\dots 5, 1, 6, 4, 7, 11, 2] \\ & & \underbrace{\hspace{1.5cm}}_{t_0} \\ \text{After } k-1 \text{ executions} & soFar_{k-1} = 20 & [\dots 5, 1, 6, 4, 7, 11, 2] \\ & & \underbrace{\hspace{1.5cm}}_{t_{k-1}} \\ \text{After } k \text{ executions} & soFar_k = 24 & [\dots 5, 1, 6, 4, 7, 11, 2] \\ & & \underbrace{\hspace{1.5cm}}_{t_k} \end{array}$$

Figure 3.1 A particular step in the execution of the **while** loop in *impSumStack*.

In general, to show that we have a loop invariant condition and that this can be used to prove correctness, three main steps are needed.

Strategy 3.1

To prove that an implementation using a while loop is correct

1. Check that the states of the variables just before the loop is entered satisfy the proposed loop invariant condition.
2. Check that a single execution of the loop body does not change the value of the proposed loop invariant.
3. Use the fact that the loop ceases execution when the **while** condition becomes **false**, together with the proven loop invariant condition, to discover information about the states of the variables just after the loop finishes execution.

The hope is that the information obtained in Step 3 will lead us to the conclusion that the value returned by the implementation must satisfy the postcondition in the specification.

In the proof discussed above, to show that the implementation was correct we first determined a condition that we believed to be a loop invariant. This invariant followed from the design of our implementation. In the proofs we shall ask you to perform, we shall not ask you to determine an appropriate loop invariant.

Because we were explaining this as an introductory example, the proof as given is both long-winded and not presented in the order that would be usual. We now turn to some examples expressed in the way we would normally lay out such a proof of correctness.

For Step 3 to be successful we must choose a suitable loop invariant. For example, if *impSumStack* had included a variable *index*, which counted the number of loop executions, $SIZE(t) + index$ would be a loop invariant. It would not be useful, however. It does not convey the purpose of the loop, and does not enable us to prove correctness. This would be an unsuitable loop invariant.

3.2 Proofs of correctness

In this section we will ask you to give some proofs of the type considered in Subsection 3.1. We will first give an example laid out in a less discursive way than the discussion of *SUMSTACK* in the previous subsection.

EXAMPLE 3.1

A specification of a function *POWERTWO* is given below, together with code *impPowerTwo* intended to implement it

function *POWERTWO*(*n* in Int) **return** in Int

pre $n \geq 0$.

post The returned value is 2^n .

```
function POWERTWO(n)
{  //impPowerTwo
  var soFar in Int
  var toDo in Int
  toDo <-- n                                //(1)
  soFar <-- 1                               //(2)
  while (NOT(toDo == 0))                   //(3)
  {
    soFar <-- soFar * 2                     //(4)
    toDo <-- toDo - 1                       //(5)
  }
  return soFar
}
```

Remember that 2^n means $2 \times 2 \times 2 \times \dots \times 2$, where there are n 2s.

We can prove that the implementation *impPowerTwo* is correct as follows. We first show that

$$soFar \times 2^{toDo} = 2^n \quad (\text{LIC})$$

is a loop invariant condition for *impPowerTwo*. To do this, let $soFar_n$ and $toDo_n$ be the values of the local variables in *impPowerTwo* after n executions of the loop body. Before the loop is entered, we have:

$$soFar_0 = 1 \quad \text{and} \quad toDo_0 = n.$$

Then $soFar_0 \times 2^{toDo_0} = 1 \times 2^n = 2^n$.

So the condition in (LIC) holds when the loop is first entered.

From the initialisations in statements (1) and (2). Before the loop is entered, the loop body has been executed 0 times.

Now, from the code in the loop body, we have, for $k > 0$:

$$soFar_k = 2 \times soFar_{k-1} \quad \text{and} \quad toDo_k = toDo_{k-1} - 1. \quad \text{From statements (4) and (5).}$$

So we have

$$soFar_k \times 2^{toDo_k} = 2 \times soFar_{k-1} \times 2^{toDo_{k-1}-1} = soFar_{k-1} \times 2^{toDo_{k-1}}.$$

This shows that $soFar \times 2^{toDo}$ is a loop invariant. So if the condition (LIC) holds after $k - 1$ executions of the loop body, then it will still hold after k executions of the loop body. We know that it holds after zero executions — upon entry to the loop. Hence, by mathematical induction, (LIC) holds after n executions of the loop body for any integer $n \geq 0$. So we have established that (LIC) is a loop invariant condition for *impPowerTwo*.

Let $toDo_F$ and $soFar_F$ be the values of the variables when the loop is left. The **while** loop will stop executing when the **while** condition in statement (3) becomes **false**. Consequently, we know that $toDo_F = 0$. So, using the loop invariant condition (LIC) when the loop is left, we have

$$2^n = soFar_F \times 2^{toDo_F} = soFar_F \times 2^0 = soFar_F \times 1 = soFar_F.$$

Now $soFar_F$ is the value returned by *impPowerTwo*. So *impPowerTwo* returns the value 2^n . Examining the specification of *POWERTWO*, we see that the function must return the value 2^n . So the implementation *impPowerTwo* returns the value given in the postcondition in the specification of *POWERTWO*, i.e. *impPowerTwo* correctly evaluates *POWERTWO*.

We now ask you to try giving a proof of correctness. The implementation we consider is based on the idea that removing the item from the top of a stack will not change the element at the bottom of the stack.

Activity 3.4

A specification of a function *BOTTOM* is given below, together with code *impBottom* intended to implement it. Prove that

$$BOTTOM(t) = BOTTOM(s) \quad (\text{LIC})$$

is a loop invariant condition for *impBottom*, and hence show that this implementation is correct.

function *BOTTOM*(*s* in Stack of **X**) **return** in **X**

pre *s* is not empty.

post The returned value is the bottom (leftmost) item in the stack *s*. For example, *BOTTOM*([1, 2, 3, 4]) = 1.

```
function BOTTOM(s)
{  //impBottom
  var t in Stack of X
  t <-- s
  while (NOT(SIZE(t) == 1))
  {
    t <-- POP(t)
  }
  return PEEK(t)
}
```

Discussion Let t_n be the value of the variable t after n executions of the loop. Before the loop is entered ($n = 0$), we have $t_0 = s$, from the statement initialising t . So

$$BOTTOM(t_0) = BOTTOM(s),$$

i.e. the proposed loop invariant condition holds upon entry to the loop.

The only statement in the loop body is $t \leftarrow POP(t)$. So $t_k = POP(t_{k-1})$. Now deleting the top item in a stack will not change the item at the bottom of the stack, unless there is only one item in the stack. The **while** condition ensures that the loop body is not executed if the stack t contains only one item. So we have, for $k > 0$:

$$BOTTOM(t_k) = BOTTOM(t_{k-1}).$$

So if the condition in (LIC) holds after $k - 1$ executions of the loop body, i.e. if

$$BOTTOM(t_{k-1}) = BOTTOM(s),$$

then we will also have

$$BOTTOM(t_k) = BOTTOM(s),$$

i.e. the condition in (LIC) still holds after k executions of the loop body. Since the condition in (LIC) holds upon entry to the loop, we can deduce, by mathematical induction, that it holds after an arbitrary number of executions of the loop body.

In particular, we know that the loop invariant condition in (LIC) holds when the loop is left. If t_F is the value of t just after the loop is left, then we have

$$BOTTOM(t_F) = BOTTOM(s). \quad (1)$$

Now the loop is left when the **while** condition becomes false. So we know that:

$$SIZE(t_F) = 1.$$

Now a stack of size one contains only one element, and so this element is both its bottom element and its top element. So we have:

$$PEEK(t_F) = BOTTOM(t_F). \quad (2)$$

From (1) and (2), we have

$$PEEK(t_F) = BOTTOM(s).$$

Now $PEEK(t_F)$ is the value returned by *impBottom*. So the value returned by *impBottom* is $BOTTOM(s)$, satisfying the specification of the function $BOTTOM$. So *impBottom* correctly evaluates the function $BOTTOM$.

This proves that $BOTTOM(t)$ is a loop invariant.

We are now justified in referring to (LIC) as a loop invariant condition — we have proved that it is one!

It is not essential when establishing a loop invariant condition to introduce notation such as t_n for the states of the variables after n executions of the loop body. The key lies in establishing that a suitable loop invariant quantity is not changed by a single execution of the loop body. In the next example we will give an argument that does not use notation such as t_n .

EXAMPLE 3.2

A specification of a function *DELSPACES* is given below, together with code *impDelSpaces* intended to implement it.

function *DELSPACES*(*v* in **Vector of Char**) **return** in **Vector of Char**
pre *v* contains at least one character that is not a space.
post The returned value is the vector formed by deleting from *v* any space characters that appear at the front of *v*.

The class **Vector** was introduced in Unit 6.

```
function DELSPACES(v)
{  //impDelSpaces
  var rest in Vector of Char
  rest <-- v.clone()
  while (ASC(rest.at(1)) == 32)
  {
    rest.removeFirst()
  }
  return rest
}
```

To show that

$$DELSPACES(v) = DELSPACES(rest) \quad (\text{LIC})$$

is a loop invariant condition for *impDelSpaces*, we can argue as follows.

Just before the loop is entered, the statement $rest \leftarrow v.clone()$ ensures that *rest* is identical to *v*, and so (LIC) will be true upon entry to the loop.

The body of the loop consists of the single statement *rest.removeFirst()*. This will delete the first character from *rest*. This statement will only be executed if the **while** condition is **true**, i.e. if the ASCII code of the character *rest.at(1)* is 32. The **while** condition will only be **true** if the first character in *rest* is a space character. So the loop body is only executed if *rest* has a space character as its first character, and the effect of the loop body is to delete that first space character. Deleting a space character from the front of *rest* will not change the value of the quantity *DELSPACES*(*rest*) (which is the vector formed by deleting all the space characters from the front of *rest*). So the quantity *DELSPACES*(*rest*) is a loop invariant. Taking the initial value of the variable *rest* into account, we can then see that

$$DELSPACES(v) = DELSPACES(rest)$$

is a loop invariant condition for *impDelSpaces*.

To complete the proof of correctness, note that the loop will be left when the **while** condition becomes **false**. So if *rest_F* is the value of *rest* when the loop is left, the first character in *rest_F* is not a space character. Now for a vector *rest_F* whose first character is not a space character, *DELSPACES*(*rest_F*) must be the same as *rest_F*. (We form *DELSPACES*(*rest_F*) by deleting from the front of *rest_F* any space characters that are there — but we know that there are no space characters in *rest_F* to be deleted.)

Using the loop invariant condition, we have

$$DELSPACES(v) = DELSPACES(rest_F).$$

From the **while** condition, we have that *DELSPACES*(*rest_F*) = *rest_F*.

Combining these, we have *DELSPACES*(*v*) = *rest_F*.

Now $rest_F$ is the value returned by *impDelSpaces*. So the value returned is $DELSPACES(v)$, i.e. the correct value as given in the specification of $DELSPACES$. So *impDelSpaces* is correct.

A proof using mathematical induction works by establishing a result about integers. The approach in Example 3.2 does not make the link to integers explicit. However, any proof establishing a loop invariant condition uses mathematical induction in the same way. The integer is the number of times that the loop body has been executed, and the use of mathematical induction can be made explicit by introducing the sort of notation for the state of each local variable after n executions of the loop body used in earlier examples.

In Example 3.2, we established the loop invariant condition by showing two things.

1. The proposed loop invariant condition holds upon entry to the loop.
2. A suitable loop invariant quantity is not changed by a single execution of the loop body.

A specification of a function *REMOVE*, and code intended to implement *REMOVE*, are given below.

function *REMOVE*(v in Vector of X , x in X) **return** in Vector of X
pre true.

post The returned value is the vector formed by removing all copies of the item x from v . So, for example, $REMOVE([3, 1, 1, 7, 12, 1], 1) = [3, 7, 12]$.

The “suitable loop invariant quantity” must contain all the local variables in the proposed loop invariant condition.

```
function REMOVE(v,x)
{  //impRemove
  var old in Vector of X
  var current in Vector of X
  old <-- v.clone()
  while (NOT(old.size() == 0))
  {
    if (NOT(old.first() == x)) then
    {
      current.addLast(old.first())
    }
    old.removeFirst()
  }
  return current
}
```

To express a loop invariant for this example, we shall use the infix operation $\oplus$ on sequences to represent concatenation. Recall that $s \oplus t$ is the sequence formed by writing as a single sequence: first the members of s , then the members of t . We first illustrate a proposed loop invariant condition for *impRemove*.

See Unit 3, p. 40.

Activity 3.5

- (a) If v is $[2, 1, 1, 3, 1, 1, 7, 12, 1, 9]$, what is $REMOVE(v, 1)$?
- (b) If old is the sequence $[1, 7, 12, 1, 9]$, and $current$ is the sequence $[2, 3]$, what is $current \oplus REMOVE(old, 1)$?

Discussion

- (a) Referring to the specification of *REMOVE*, we see that $REMOVE(v, 1)$ is formed by removing all copies of 1 from the sequence v , so

$$REMOVE([2, 1, 1, 3, 1, 1, 7, 12, 1, 9], 1) = [2, 3, 7, 12, 9].$$

- (b) $REMOVE(old, 1)$ is formed by removing all copies of 1 from the sequence old , so $REMOVE([1, 7, 12, 1, 9], 1) = [7, 12, 9]$. Then

$$current \oplus REMOVE(old, 1) = [2, 3] \oplus [7, 12, 9] = [2, 3, 7, 12, 9].$$

Activity 3.6

Explain why

$$REMOVE(v, x) = current \oplus REMOVE(old, x)$$

is a loop invariant condition for *impRemove*, and hence show that the implementation *impRemove* is correct.

Discussion Before the loop is entered, the following occurs.

- old is set to a copy of v .
- $current$ is initialised in the default state, the empty vector.

So just before the loop is entered, we have

$$current \oplus REMOVE(old, x) = [] \oplus REMOVE(v, x) = REMOVE(v, x).$$

So the proposed loop invariant condition holds as the loop is entered.

Now what is the effect of one execution of the loop body on the quantity $current \oplus REMOVE(old, x)$? There are two cases, depending on whether the **if** condition is **true** or **false**.

1. If the first element in old is x , then only the statement $old.removeFirst()$ is executed. The result is that the first element is deleted from old (with no other changes).
2. If the first element in old is not x , then this element is deleted from old , and is also added to the end of $current$ by the statement following **then**.

The sequence $REMOVE(old, x)$ will not contain any copies of x . So deleting x from the start of old in case 1 will not change the value of $REMOVE(old, x)$. In this case, the value of $current$ is not changed by the loop body, so the value of $current \oplus REMOVE(old, x)$ will not change.

In case 2, the first element in old will appear in $REMOVE(old, x)$. Deleting this element from the start of old will remove it from $REMOVE(old, x)$. So deleting the first element of old and adding this same element to the end of $current$ will not change the joined sequence $current \oplus REMOVE(old, x)$. All that has happened is that one element has been moved from $REMOVE(old, x)$ to $current$.

So we have shown that the quantity $current \oplus REMOVE(old, x)$ is not changed by a single execution of the body of the loop in *impRemove*, and so is a loop invariant. Since the condition holds as the loop is entered, we then have the loop invariant condition:

$$current \oplus REMOVE(old, x) = REMOVE(v, x).$$

This holds however many times the loop has been executed, and so, in particular, holds when the loop is left.

Let old_F and $current_F$ be the values of the variables when the loop is left. Then we have, from the loop invariant condition:

$$current_F \oplus REMOVE(old_F, x) = REMOVE(v, x).$$

Now the loop is left when the **while** condition becomes **false**, i.e. when old becomes empty. So $old_F = []$ and thus we have:

$$REMOVE(v, x) = current_F \oplus REMOVE(old_F, x) = current_F \oplus REMOVE([], x).$$

Removing all the copies of x from an empty vector will leave an empty vector, so $REMOVE([], x)$ is the empty vector. Therefore we have:

$$REMOVE(v, x) = current_F \oplus [] = current_F.$$

Now $current_F$ is the value returned by the code in *impRemove*. So *impRemove* returns the value $REMOVE(v, x)$, satisfying the specification of *REMOVE*. Thus *impRemove* is a correct implementation of *REMOVE*.

You may have noticed a change in the style of proof in this section compared with those in Section 2. Our arguments about specifications and implementations may use a more relaxed terminology than the formal mathematical proofs in Section 2. This observation applies in general to proofs in software engineering or programming when compared with mathematical proofs.

So far, we have used loop invariants to prove the correctness of an implementation that was based on a **while** loop. We can use the same method to find errors in code, such as an inappropriate **while** condition or variable initialisation. Such errors may not always be found by testing, as only a limited number of cases can be checked. Also, this method may help to locate the errors, whereas testing will at best show that there is some error in the implementation.

Activity 3.7

The function *SUMSQ*(n) returns the sum of the squares of the integers from 1 to n . A specification of *SUMSQ* is given below, as is code intended to evaluate it.

function *SUMSQ*(n in Int) **return** in Int

pre $n \geq 1$.

post The returned value is the sum of the squares of the integers from 1 to n , i.e. $1^2 + 2^2 + 3^2 + \dots + n^2$.

```
function SUMSQ(n)
{
    // impSumSqX
    var next in Int
    var soFar in Int
    soFar <-- 0
    next <-- 1
    while (NOT(next == n))
    {
        next <-- next + 1
        soFar <-- soFar + next*next
    }
    return soFar
}
```


- (a) By reference to the specification of *SUMSQ*, explain why, for any $n > 1$, we have that:

$$SUMSQ(n) = SUMSQ(n-1) + n^2.$$

- (b) Show that $SUMSQ(next) - soFar$ is a loop invariant for the loop body in *impSumSqX*.
 (c) Does the condition $SUMSQ(next) - soFar = 0$ hold upon entry to the loop?
 (d) If $SUMSQ(next) - soFar = 0$ held when the loop is left, would we then be able to deduce that the implementation returns the correct value?
 (e) Identify the error in the code in *impSumSqX* and correct it.

Discussion

- (a) The sum of the squares of the integers from 1 to n is the sum of the squares of the integers from 1 to $n-1$, plus the square of n . So

$$SUMSQ(n) = SUMSQ(n-1) + n^2.$$

- (b) With the usual notation for the values of the variables after n executions of the loop body, we can see that a single execution of the loop body in *impSumSqX* changes the variable values as follows.

$$\begin{aligned} next_k &= next_{k-1} + 1 \\ soFar_k &= soFar_{k-1} + (next_k)^2 \end{aligned}$$

Then

$$\begin{aligned} SUMSQ(next_k) - soFar_k &= SUMSQ(next_{k-1} + 1) - (soFar_{k-1} + (next_k)^2) \\ &= SUMSQ(next_{k-1}) + (next_{k-1} + 1)^2 - (soFar_{k-1} + (next_{k-1} + 1)^2) \quad (*) \\ &= SUMSQ(next_{k-1}) - soFar_{k-1}. \end{aligned}$$

Line (*) used the point noted in (a) above with n equal to $next_{k-1} + 1$.

Thus $SUMSQ(next) - soFar$ is a loop invariant, since this quantity is not changed by a single execution of the loop body.

- (c) No. Upon entry to the loop, we have $next_0 = 1$ and $soFar_0 = 0$. So $SUMSQ(next_0) = SUMSQ(1) = 1$, and we do not have $SUMSQ(next_0) - soFar_0 = 0$.
 (d) Let $next_F$ and $soFar_F$ be the values of the variables when the loop is left. Suppose that

$$SUMSQ(next_F) - soFar_F = 0 \quad (1)$$

were to be true. The loop will stop executing when the **while** condition becomes **false**, i.e. when

$$next_F = n.$$

Combining this with (1) would give $SUMSQ(n) - soFar_F = 0$, i.e.

$$SUMSQ(n) = soFar_F.$$

The code in *impSumSqX* returns the value $soFar_F$. So the correct value would be returned if $SUMSQ(next_F) - soFar_F = 0$ were true.

- (e) The only reason why we cannot complete a proof of correctness here is that the proposed loop invariant condition does not hold when the loop is entered, as shown in (c). This tells us that the wrong initial value has been given to one or more of the variables. We know from (b) that the code in the loop body preserves the proposed loop invariant quantity.

We know from (d) that were the condition to hold when the loop is left, then the correct value would be returned. The only correction required is to change the statement

$soFar \leftarrow 0$

to become

$soFar \leftarrow 1$.

We will then have, upon entry to the loop, that $next_0 = 1$ and $soFar_0 = 1$. Then $SUMSQ(next_0) = SUMSQ(1) = 1$, and we will have

$SUMSQ(next_0) - soFar_0 = 0$.

Equally well, we could change the initial value given to $next$, so that the initialisations are:

$soFar \leftarrow 0$

$next \leftarrow 0$.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: loop invariant, loop invariant condition.
- ☐ Show that a given quantity is a loop invariant.
- ☐ Given a loop invariant condition, prove the correctness of an implementation involving a **while** loop.
- ☐ Find certain errors in code and correct them by using the same approach as for proving correctness.

Exercises on Section 3

Exercise 3.1

A specification of a function *FACTORIAL* is given below, followed by code intended to implement it. Prove that

$soFar \times FACTORIAL(todo) = FACTORIAL(n)$

is a loop invariant condition for *impFact*. Hence prove that this implementation is correct.

function *FACTORIAL*(*n in Int*) **return in Int**

pre $n > 0$.

post The returned value is the result of multiplying together all the integers from 1 to n , i.e. $1 \times 2 \times 3 \times \dots n$.

```
function FACTORIAL(n)
{
  //impFact
  var todo in Int
  var soFar in Int
  soFar <-- 1
  todo <-- n
  while (NOT(todo == 1))
  {
    soFar <-- soFar * todo
    todo <-- todo - 1
  }
  return soFar
}
```

Exercise 3.2

A function *FIRSTAT*, which gives the position of the first occurrence of a given character in a vector of characters, is specified below.

function *FIRSTAT*(*c* in **Char**, *t* in **Vector of Char**) **return** in **Int**

pre The character *c* appears at least once in the vector *t*.

post The returned value is *i*, where the character in *t* with index *i* is *c*, and none of the characters in *t* with an index smaller than *i* is *c*.

For example, we have *FIRSTAT*('a', ['b', 'a', 'n', 'a', 'n', 'a']) = 2.

- (a) Suppose that *c* is a character, and *t* is a variable of type **Vector of Char** that satisfies the precondition of *FIRSTAT*. Let *v* be a vector whose state is that of *t*, but with the first character removed. By referring to the postcondition of *FIRSTAT*, explain why the following is true.

$$\begin{aligned} FIRSTAT(c, t) &= 1 \text{ if } c == t.first() \\ FIRSTAT(c, t) &= 1 + FIRSTAT(c, v) \text{ otherwise} \end{aligned}$$

- (b) A proposed implementation (*impFirstAt*) of *FIRSTAT* is given below. Prove that $FIRSTAT(c, t) = p + FIRSTAT(c, s)$ is a loop invariant condition for *impFirstAt*.

```
function FIRSTAT(c,t)
{
    // impFirstAt
    var p in Int
    var s in Vector of Char
    p <-- 0
    s <-- t.clone()
    while (NOT(c == s.first()))
    {
        p <-- p + 1
        s.removeFirst()
    }
    return p + 1
}
```

- (c) Hence prove that *impFirstAt* is a correct implementation of *FIRSTAT*.

Exercise 3.3

The code below is the attempt at an implementation of the function *INTSUM* given in the introduction to this unit.

```
function INTSUM(n)
{
    // sumTry
    var nextNum in Int
    var soFar in Int
    soFar <-- 0
    nextNum <-- n
    while (NOT(nextNum == 0))
    {
        nextNum <-- nextNum - 1
        soFar <-- soFar + nextNum
    }
    return soFar
}
```

- (a) Trace the code in *sumTry* when $n = 3$. Determine the value returned in this case, and hence show that the implementation is not correct.
- (b) Does the condition
$$soFar + INTSUM(nextNum) = INTSUM(n) \quad (LIC)$$
hold for the values of the variables *soFar* and *nextNum* before the loop is entered?
- (c) If the condition in (LIC) did hold when the loop stops executing, could we prove that *sumTry* is correct?
- (d) What effect does the first execution of the loop body have on the quantity $soFar + INTSUM(nextNum)$? Deduce that (LIC) is not a loop invariant condition for *sumTry*.
- (e) Modify the algorithm *sumTry* to an algorithm that is correct, and for which $soFar + INTSUM(nextNum) = INTSUM(n)$ is a loop invariant condition.

4 Other proofs using induction

In this section we will look at three examples of how mathematical induction can be used to prove a variety of results. The three examples in this section will provide more practice in reading formal arguments about software and mathematical problems.

We saw in Section 3 how we could prove the correctness of code involving a **while** loop. In the first example below we will consider code involving a **for** loop. Some modifications to our method are needed.

EXAMPLE 4.1

Consider the following fragment of code.

```
var aSpot in Spot
var index in Int
for (index <-- 1 to n)
{
  aSpot.upOne()
  aSpot.leftOne()
}
```

It looks likely that, for $n \geq 1$, the final state of $aSpot$ is $(-n, n)$, but how might we prove this assertion?

We shall use the notation $aSpot_j$ for the state of $aSpot$ after j executions of the loop body. (Note that $aSpot_j$ is a state, not a variable identifier.) Consider the following assertion.

$p(j) : aSpot_j$ is $(-j, j)$.

We will prove, using mathematical induction, that $p(j)$ holds for $0 \leq j \leq n$. The induction is again based on the number of times the loop body has been executed.

We will take $j = 0$ as the starting value for the induction. For $j = 0$, the loop body has not been executed at all, and so $aSpot_0$ is the default position of $aSpot$, which is $(0, 0)$. For $j = 0$, $(-j, j)$ is $(0, 0)$, so we see that $p(0)$ holds.

Now suppose that $p(k-1)$ holds, i.e.

$aSpot_{k-1}$ is $(-(k-1), k-1)$.

After a further execution of the loop body, $aSpot$ will have been moved up by one and left by one, resulting in

$aSpot_k$ being $(-(k-1) - 1, k-1 + 1) = (-k, k)$;

that is, $p(k)$ holds. So we have shown that if $p(k-1)$ holds then $p(k)$ also holds. This proves the inductive step.

So we have shown the following.

(B) $p(0)$.

(I) If $p(k-1)$ holds then $p(k)$ holds.

Thus, by mathematical induction, it follows that $aSpot_j = (-j, j)$ for any integer j with $0 \leq j \leq n$. In the given fragment, the loop body is executed n times, and so the final state of $aSpot$ is $aSpot_n = (-n, n)$.

EXAMPLE 4.2

We next consider a result from set theory. You met the set of subsets of a set X in Unit 7. We stated there that if X has cardinality n then the cardinality of the set of subsets of X , $SetOfX$, is 2^n . We are now able to prove this result by using mathematical induction.

See Unit 7, p. 24.

To prove that the set $SetOfX$ has cardinality 2^n , where X has cardinality n , for $n \geq 1$, we use induction on n , starting at 1.

When $n = 1$, the set X contains one element. Then $SetOfX$ contains just two members: the empty set and the set X itself, and so has cardinality 2. Now $2^1 = 2$, as required for the starting step.

Now suppose that for any set Y of cardinality $k - 1$ (where $k > 1$), $SetOfY$ has cardinality 2^{k-1} , and consider a set X that contains k elements. Let x be one of the elements in X and note that the set difference $X - \{x\}$ is a set of cardinality $k - 1$. Then the set of subsets of $X - \{x\}$ has cardinality 2^{k-1} by our induction hypothesis.

Now every subset of X either contains x or does not contain x . The subsets of X that do not contain x are precisely the subsets of $X - \{x\}$, of which, from the induction hypothesis, there are 2^{k-1} . The subsets of X that do contain x can be found by adding x to each subset of $X - \{x\}$. Consequently there are 2^{k-1} of these, also. Thus X has in total $2^{k-1} + 2^{k-1}$ subsets, which proves that the set $SetOfX$ has cardinality

$$2^{k-1} + 2^{k-1} = 2 \times 2^{k-1} = 2^k.$$

This completes the inductive step, and so we can deduce by mathematical induction that, for a set X of cardinality n , (where $n \geq 1$), the set $SetOfX$ has cardinality 2^n .

The result in Example 4.2 also holds for an empty set. An empty set has cardinality 0, and $2^0 = 1$. Now an empty set has exactly one subset — the empty set $\{\}$ itself, and so $SetOfX$ has cardinality 1 in this case, as required.

Our final example involves a **for** loop, like that in Example 4.1. Attempt to prove the result for yourself, but do not spend too long on it.

We could have started the argument in Example 4.2 with this case, but such special cases are sometimes a little confusing.

Activity 4.1

A fragment of code is given below.

```
var myTurtle in Turtle
var i in Int
for (i <-- 1 to n)
{
  myTurtle.forward(2*i)
  myTurtle.leftTurn()
  myTurtle.leftTurn()
}
```

Experiments with the WorkPad for various values of n suggest that after this fragment has been executed, the marker state of *myTurtle* is as follows.

- If n is odd, the marker state is $(n + 1, 0, \text{left}(180))$.
- If n is even, the marker state is $(-n, 0, \text{right}(0))$.

Use mathematical induction to prove that this must indeed be the case for any integer n with $n \geq 1$.

Discussion For an integer j with $1 \leq j \leq n$, let the state of *myTurtle* after j executions of the body of the loop be *myTurtle_j*. Let $p(j)$ be the assertion that:

1. if j is odd, the marker state is $(j + 1, 0, \text{left}(180))$;
2. if j is even, the marker state is $(-j, 0, \text{right}(0))$.

Our induction will start at $j = 1$, so first consider this case.

With $j = 1$, the body of the **for** loop in the given fragment has been executed once. This will leave the marker of *myTurtle* in the state $(2, 0, \text{left}(180))$.

Since 1 is odd, the assertion $p(1)$ is given by item 1 above, so $p(1)$ is:

the marker state is $(1 + 1, 0, \text{left}(180))$.

So we have shown that $p(1)$ holds.

Now suppose that $p(k - 1)$ holds. We need to show that $p(k)$ must then also hold. There are two cases, depending on whether $k - 1$ is odd or even. We need first to note that in the k th execution of the loop body the loop variable i has the value k (because i starts at 1).

Case 1: $k - 1$ is odd.

In this case, the state of the marker of *myTurtle* after $k - 1$ executions of the loop body is given by substituting $j = k - 1$ into the expression $(j + 1, 0, \text{left}(180))$, and so is $(k, 0, \text{left}(180))$. If the loop body is executed one further time, the following occurs.

1. The marker is moved $2k$ places left — the current marker direction. This will move the marker from the position $(k, 0)$ to the position $(-k, 0)$.
2. The marker is rotated left twice, and so is turned through 180 degrees. This will change the marker direction from $\text{left}(180)$ to $\text{right}(0)$.

So after k executions of the loop body, the state of the marker of *myTurtle* is $(-k, 0, \text{right}(0))$. If $k - 1$ is odd then k is even, and this is as required by $p(k)$ when k is even.

Case 2: $k - 1$ is even

In this case, the state of the marker of *myTurtle* after $k - 1$ executions of the loop body is given by substituting $j = k - 1$ into the expression $(-j, 0, \text{right}(0))$, and so is $(-(k - 1), 0, \text{right}(0))$. If the loop body is executed one further time, the following occurs.

1. The marker is moved $2k$ places right — the current marker direction. This will move the marker from the position $(-(k - 1), 0)$ to the position $(k + 1, 0)$.
2. The marker is rotated left twice, and so is turned through 180 degrees. This will change the marker direction from $\text{right}(0)$ to $\text{left}(180)$.

So after k executions of the loop body, the state of the marker of *myTurtle* is $(k + 1, 0, \text{left}(180))$. If $k - 1$ is even, then k is odd, so this is as required by $p(k)$ when k is odd.

We have now shown the following.

(B) $p(1)$ holds.

(I) If $p(k - 1)$ holds then $p(k)$ holds.

We can conclude, by mathematical induction, that $p(j)$ holds for all integers j between 1 and n . In the given fragment, the loop body is executed n times, so the final state of *myTurtle* is indeed as given in the proposed formula.

Objectives for Section 4

After studying this section you should be able to do the following.

- Appreciate that mathematical induction is a versatile proof technique applicable to many different situations.

Summary of Unit 14

1. Mathematical induction provides a technique for proving a result in a situation where the hypothesis that we want to prove can be seen as a statement about integers. We can, however, apply the technique to a wide variety of situations, so long as we have some way of relating the situation to one concerning integers, for instance by considering the length of a sequence or the depth of a tree.
2. Mathematical induction can be used to prove that a given formula is a solution of a recurrence system. A formal statement of this proof technique was given, in several versions, which generalised the nature of the starting condition and the induction hypothesis.
3. We saw how to prove the correctness of code involving a **while** loop. To this end we established that a given quantity, called a loop invariant, remained unchanged by a single execution of the loop body. To establish that a proposed loop invariant condition must hold, we must also first check that it holds just before the loop is entered. Then mathematical induction enables us to show that it holds however many times the body of the loop has been executed.

Once a loop invariant condition has been established, we saw how to explain why a piece of code must correctly implement a specification. We also saw how the same technique can be used to locate errors in code.

4. Mathematical induction can be used in a variety of situations, and a few more illustrations were given.

Further Reading

Should you wish to read more about proof in computing, we give below a couple of references.

You can read more about proof as a social process in the article: *Social Processes and Proofs of Theorems and Programs* by Richard de Milo, Richard Lipton and Alan Perlis in the journal *Communications of the ACM*, Vol. 22, No. 5, May 1979, pp. 271–280.

You can extend your knowledge of proof by induction and its use in computer science by reading the first chapter of *Introduction to Automata Theory, Languages, and Computation* by John E. Hopcroft, Rajeev Motwani and Jeffrey D. Ullman; Addison-Wesley, 2001, ISBN 0-201-44124-1.

Solutions to the exercises

Solution 2.1

Let $F(n) = 5n - 6$. With $n = 2$, this formula gives $F(2) = 5 \times 2 - 6 = 4$.

This is the same value as in the initial condition, $U(2) = 4$, of the recurrence system. So the formula gives $U(2)$ correctly.

Now suppose, for $k > 2$, that the formula F gives $U(k - 1)$ correctly, so that

$$U(k - 1) = F(k - 1) = 5(k - 1) - 6 = 5k - 11. \quad (\text{H})$$

Then, for $k > 2$,

$$\begin{aligned} U(k) &= U(k - 1) + 5 && (\text{from the recurrence system}) \\ &= 5k - 11 + 5 && (\text{from H}) \\ &= 5k - 6 \\ &= F(k). \end{aligned}$$

Thus for all $k > 2$, if the formula F gives $U(k - 1)$ correctly, then it also gives $U(k)$ correctly, so establishing the inductive step.

Having shown that the starting step and the inductive step hold, we can deduce, using the principle of mathematical induction, that the formula $F(n) = 5n - 6$ gives the same values for all $n \geq 2$ as the function $U(n)$ defined by the recurrence.

Solution 2.2

Let $F(n) = 8 \times 2^n - 3$. With $n = 0$, this formula gives

$$F(0) = 8 \times 2^0 - 3 = 8 - 3 = 5.$$

This is the same value as in the initial condition, $T(0) = 5$, of the recurrence system. So the formula gives $T(0)$ correctly.

Now, for $k > 0$, suppose that the formula F gives $T(k - 1)$ correctly, so that

$$T(k - 1) = F(k - 1) = 8 \times 2^{k-1} - 3. \quad (\text{H})$$

Then, for $k > 0$,

$$\begin{aligned} T(k) &= 2T(k - 1) + 3 && (\text{from the recurrence system}) \\ &= 2 \times (8 \times 2^{k-1} - 3) + 3 && (\text{from H}) \\ &= 2 \times 8 \times 2^{k-1} - 6 + 3 \\ &= 8 \times 2 \times 2^{k-1} - 3 \\ &= 8 \times 2^k - 3 \\ &= F(k). \end{aligned}$$

Thus for any integer $k > 0$, if the formula F gives $T(k - 1)$ correctly, then it also gives $T(k)$ correctly.

Having shown that the starting step and the inductive step hold, we can deduce, using the principle of mathematical induction, that the formula $8 \times 2^n - 3$ gives the same values for all $n \geq 0$ as does the function $T(n)$ defined by the recurrence.

Solution 2.3

(a) To obtain the initial condition, we set $n = 1$. The “integers from 1 to 1” just involves a single integer, and so the sum just involves one term in this case, which is $1^2 = 1$. So $SUMSQ(1) = 1$.

To express the recursive part, note the following. The sum of squares for the integers from 1 to n is the sum of squares for the integers from 1 to $n - 1$ with the term for n (that is, n^2) added. Putting these together gives the recurrence system below.

$$\begin{aligned} SUMSQ(1) &= 1 & (S) \\ SUMSQ(n) &= SUMSQ(n-1) + n^2 \quad (n > 1) & (R) \end{aligned}$$

(b) Let $F(n) = \frac{n(n+1)(2n+1)}{6}$.

$\frac{n(n+1)(2n+1)}{6}$ may be written as $\frac{1}{6}n(n+1)(2n+1)$.

With $n = 1$, this formula gives $F(1) = \frac{1 \times 2 \times 3}{6} = 1$.

This is the same as the initial condition, $SUMSQ(1) = 1$, of the recurrence system; that is,

$$F(1) = SUMSQ(1).$$

Now, for $k > 1$, suppose that the formula F gives $SUMSQ(k-1)$ correctly, so that

$$SUMSQ(k-1) = F(k-1) = \frac{(k-1)(k-1+1)(2(k-1)+1)}{6} = \frac{(k-1)k(2k-1)}{6}. \quad (H)$$

Then for $k > 1$,

$$SUMSQ(k) = SUMSQ(k-1) + k^2 \quad (\text{from the recurrence system})$$

$$= \frac{(k-1)k(2k-1)}{6} + k^2 \quad (\text{from H})$$

$$= \frac{k(k-1)(2k-1) + 6k^2}{6}$$

$$= \frac{k((k-1)(2k-1) + 6k)}{6} \quad (\text{taking out a factor } k)$$

$$= \frac{k(2k^2 - 3k + 1 + 6k)}{6}$$

$$= \frac{k(2k^2 + 3k + 1)}{6}.$$

So $SUMSQ(k) = \frac{1}{6}k(2k^2 + 3k + 1)$. We want to show that this expression is equal to $F(k) = \frac{1}{6}k(k+1)(2k+1)$. To see this, we can multiply out the last two brackets in $F(k)$ and obtain $F(k) = \frac{1}{6}k(2k^2 + 3k + 1)$.

It follows that $F(k) = SUMSQ(k)$.

So, for $k > 1$, if the formula F gives $SUMSQ(k-1)$ correctly, then it also gives $SUMSQ(k)$ correctly.

Having shown that the starting step and the inductive step hold, we can deduce, using the principle of mathematical induction, that the formula $\frac{1}{6}n(n+1)(2n+1)$ gives the same values for all $n \geq 1$ as does the function $SUMSQ(n)$ defined by the recurrence.

Solution 2.4

(a) A full binary tree of depth 1 consists only of a root node. So

$$NODES(1) = 1.$$

For $n > 1$, the number of nodes in a full binary tree of depth n is the sum of the number of nodes in a full binary tree of depth $n - 1$ and the number of leaves in a full binary tree of depth n . We showed in Activity 2.3 that $LEAVES(n) = 2^{n-1}$. Thus

$$NODES(n) = NODES(n - 1) + 2^{n-1} \quad (n > 1).$$

This shows that $NODES(n)$ satisfies the given recurrence system.

We can see that $NODES(n)$ must be a sum as follows.

$$\begin{aligned} NODES(n) &= NODES(n - 1) + 2^{n-1} \\ NODES(n - 1) &= NODES(n - 2) + 2^{n-2} \\ NODES(n - 2) &= NODES(n - 3) + 2^{n-3} \\ &\vdots \\ NODES(3) &= NODES(2) + 2^2 \\ NODES(2) &= NODES(1) + 2 \\ NODES(1) &= 1. \end{aligned}$$

Putting these together gives

$$NODES(n) = 1 + 2 + 2^2 + \dots + 2^{n-3} + 2^{n-2} + 2^{n-1}.$$

This is the sum of the values of 2^i from $i = 0$ to $i = n - 1$, and can be found using the formula $\frac{r^{n+1} - 1}{r - 1}$ with $r = 2$ and n replaced by $n - 1$.

$$\text{We obtain } NODES(n) = \frac{2^n - 1}{2 - 1} = 2^n - 1.$$

(b) As in (a), $NODES(1) = 1$. For a full binary tree of depth n (where $n > 1$), each of its left and right subtrees is a full binary tree of depth $n - 1$. There is also another node at the root. So we have the following.

$$\begin{aligned} NODES(1) &= 1 & (S) \\ NODES(n) &= 2 \times NODES(n - 1) + 1 \quad (n > 1) & (R) \end{aligned}$$

Let $F(n) = 2^n - 1$. With $n = 1$ this formula gives: $F(1) = 2^1 - 1 = 2 - 1 = 1$, as required by the initial condition in (S) above.

Let $k > 1$ and suppose that F gives $NODES(k - 1)$ correctly, so that

$$NODES(k - 1) = F(k - 1) = 2^{k-1} - 1. \quad (H)$$

Then, for $k > 1$

$$\begin{aligned} NODES(k) &= 2 \times NODES(k - 1) + 1 && \text{(from the recurrence)} \\ &= 2 \times (2^{k-1} - 1) + 1 && \text{(from H)} \\ &= 2^k - 2 + 1 \\ &= 2^k - 1 \\ &= F(k). \end{aligned}$$

So, for $k > 1$, if the formula F gives $NODES(k - 1)$ correctly, then it also gives $NODES(k)$ correctly. Having shown that the starting step and the inductive step hold, we can deduce, using the principle of mathematical induction, that the formula $2^n - 1$ gives the same values for all $n \geq 1$ as does the function $NODES(n)$ defined by the recurrence.

Solution 3.1

Let $soFar_k$ and $toDo_k$ be the values of the local variables in *impFact* after k executions of the loop body. Before the loop is entered, we have the following.

$$\begin{aligned} soFar_0 &= 1 \\ toDo_0 &= n \end{aligned}$$

So

$$soFar_0 \times FACTORIAL(toDo_0) = 1 \times FACTORIAL(n) = FACTORIAL(n),$$

as required.

Now suppose that the proposed loop invariant condition holds after $k - 1$ executions of the loop body, i.e.

$$soFar_{k-1} \times FACTORIAL(toDo_{k-1}) = FACTORIAL(n). \quad (1)$$

From the code in the body of the loop, we have the following.

$$\begin{aligned} soFar_k &= soFar_{k-1} \times toDo_{k-1} \\ toDo_k &= toDo_{k-1} - 1 \end{aligned}$$

Then

$$soFar_k \times FACTORIAL(toDo_k) = soFar_{k-1} \times toDo_{k-1} \times FACTORIAL(toDo_{k-1} - 1). \quad (2)$$

Now for any integer $r > 1$, we have, from the semantics of *FACTORIAL*, that

$$FACTORIAL(r) = r \times FACTORIAL(r - 1). \quad (3)$$

So from (2) and (3), with $r = toDo_{k-1}$, we have

$$\begin{aligned} soFar_k \times FACTORIAL(toDo_k) &= soFar_{k-1} \times FACTORIAL(toDo_{k-1}) \\ &= FACTORIAL(n). \end{aligned} \quad (\text{from (1)})$$

Thus, if the proposed loop invariant condition holds after $k - 1$ executions of the loop body then it will hold after k executions of the loop body. We showed above that it holds after zero executions of the loop body. Hence, by mathematical induction on the number of times the loop body has been executed, the proposed loop invariant condition does hold, however many times the loop body has been executed.

In particular, it holds when the loop stops executing. Let the value of the variables just after the loop is left be $soFar_F$ and $toDo_F$. From the loop invariant condition, we have:

$$soFar_F \times FACTORIAL(toDo_F) = FACTORIAL(n).$$

The loop will stop executing when the **while** condition becomes **false**, so we can see that $toDo_F = 1$. Now, from the specification, $FACTORIAL(1) = 1$, and so

$$\begin{aligned} FACTORIAL(n) &= soFar_F \times FACTORIAL(toDo_F) \\ &= soFar_F \times FACTORIAL(1) \\ &= soFar_F \times 1 \\ &= soFar_F. \end{aligned}$$

Now, $soFar_F$ is the value returned by *impFact*. So the value returned by *impFact* is $FACTORIAL(n)$, which is the value given in the postcondition in the specification. Hence *impFact* is proved to be a correct implementation of *FACTORIAL*.

Solution 3.2

(a) Let us first consider the case when $c == t.first()$ is true, in which the first character of t is c . The postcondition of the function *FIRSTAT* shows that *FIRSTAT* returns the position of the first occurrence of c in t . Clearly in this case the first occurrence of c is at position 1, so $FIRSTAT(c, t) = 1$.

Now suppose that c is not the first character of t and let v be a vector whose state is that of t after the first character is removed. Then the vector t differs from v in having one additional item at the front, and that item is not equal to c . So the first occurrence of c in t will be reflected in the first occurrence of c in v , where c appears one place nearer to the front of v — due to the removal of the first character from t to obtain v . For example, consider $FIRSTAT('n', ['b', 'a', 'n', 'a', 'n', 'a'])$.

$$\begin{array}{ccc} \text{first 'n' in } t & & \\ \downarrow & & \text{so } FIRSTAT('n', t) = 3 \\ t = ['b', 'a', 'n', 'a', 'n', 'a'] & & \end{array}$$

The vector v is formed by removing the first item from t .

$$\begin{array}{ccc} v = ['a', 'n', 'a', 'n', 'a'] & & \\ \uparrow & & \text{so } FIRSTAT('n', v) = 2 \\ \text{first 'n' in } v & & \end{array}$$

Hence

$$FIRSTAT(c, t) = 1 + FIRSTAT(c, v).$$

(b) Let p_k and s_k denote the states of the variables after k executions of the loop body. Upon entry to the loop, p has initial value 0 and s has the same state as t . So

$$p_0 + FIRSTAT(c, s_0) = 0 + FIRSTAT(c, t) = FIRSTAT(c, t).$$

This establishes the proposed loop invariant condition upon entry to the loop.

We now show that the quantity $p + FIRSTAT(c, s)$ is a loop invariant for *impFirstAt*.

Suppose that the loop body has been executed $k - 1$ times, and consider the k th execution. For this k th execution to occur, $s_{k-1}.first()$ cannot be equal to c — otherwise the **while** condition would have been **false** after $k - 1$ executions of the loop body and the loop would have been left. Now, from the second statement in the loop body, s_k is formed from the vector s_{k-1} by removing its first item. So, from (a), it follows that

$$FIRSTAT(c, s_{k-1}) = 1 + FIRSTAT(c, s_k).$$

Also, from the first code statement in the loop body,

$$p_k = p_{k-1} + 1.$$

So

$$p_k + FIRSTAT(c, s_k) = (p_{k-1} + 1) + (FIRSTAT(c, s_{k-1}) - 1) = p_{k-1} + FIRSTAT(c, s_{k-1}).$$

This shows that $p + FIRSTAT(c, s)$ is a loop invariant for *impFirstAt*.

It follows, by induction, that we have the following loop invariant condition.

$$p + FIRSTAT(c, s) = FIRSTAT(c, t).$$

(c) Upon exit from the loop, the **while** condition is **false**, so that, if s_F is the state of s just after the loop is left, the first character in s_F must be c . Then, from part (a), $FIRSTAT(c, s_F) = 1$. So just after leaving the loop, the variable states p_F and s_F satisfy

$$FIRSTAT(c, t) = p_F + FIRSTAT(c, s_F) = p_F + 1.$$

Now *impFirstAt* returns $p_F + 1$, which we have just shown to equal $FIRSTAT(c, t)$. It follows that *impFirstAt* is a correct implementation of the function *FIRSTAT*.

Solution 3.3

(a) A trace with n equal to 3 is given below.

Comment	$nextNum$	$soFar$	$NOT(nextNum == 0)$
Before loop	3	0	true
Enter loop	2	2	true
	1	3	true
Loop end	0	3	false

The returned value is $2 + 1 + 0 = 3$, and not $3 + 2 + 1 = 6$ as required by the specification of *INTSUM*. Hence this implementation is not correct.

(b) Before the loop is entered, $nextNum$ is n and $soFar$ has the value 0. So upon entry to the loop:

$$soFar_0 + INTSUM(nextNum_0) = 0 + INTSUM(n) = INTSUM(n).$$

Thus the given condition does hold upon entry to the loop.

(c) The loop will be left when the **while** condition becomes **false**, i.e. when $nextNum$ is 0. If the condition (LIC) holds when the loop is left, then the values of the variables upon leaving the loop, $nextNum_F$ and $soFar_F$, would satisfy the following.

$$\begin{aligned} soFar_F + INTSUM(nextNum_F) &= INTSUM(n) && \text{(from LIC)} \\ nextNum_F &= 0 && \text{(from the while condition)} \end{aligned}$$

The value returned by *sumTry* is $soFar_F$, and from the above equations we have $soFar_F + INTSUM(0) = INTSUM(n)$.

Since $INTSUM(0) = 0$ this would mean $soFar_F = INTSUM(n)$ as required. So if (LIC) were to hold when the loop is left, then we could complete a proof that *sumTry* is correct.

(d) After the loop body has been executed once, the values of the variables will be $nextNum_1 = n - 1$ and $soFar_1 = 0 + nextNum_1 = 0 + (n - 1) = n - 1$. Thus the value of the quantity $soFar + INTSUM(nextNum)$ will then be

$$soFar_1 + INTSUM(nextNum_1) = n - 1 + INTSUM(n - 1).$$

This is not the same as $INTSUM(n)$, since from the specification of *INTSUM* we have that $INTSUM(n) = INTSUM(n - 1) + n$.

This shows that $soFar + INTSUM(nextNum)$ is not a loop invariant for *sumTry*, and (LIC) does not hold.

The problem is that a single execution of the loop body does not leave $soFar + INTSUM(nextNum)$ unchanged. For example, the first execution:

1. changes $nextNum$ from n to $n - 1$;
2. adds $n - 1$ to $soFar$.

The effect of step 1 on $INTSUM(nextNum)$ is to subtract n from it. So the net effect of these two steps on $soFar + INTSUM(nextNum)$ is to subtract n and add $n - 1$. These two effects will not balance out, and therefore the quantity $soFar + INTSUM(nextNum)$ is changed by a single execution of the loop body.

(e) How can we put this right? You have probably realised by now that the problem is caused by the order of the statements in the loop body. We need to decrease $nextNum$ after incrementing $soFar$. This is achieved by exchanging the two statements in the loop body. Part (b) indicates that we have suitable initialisations for the variables, and part (c) shows that we have a suitable **while** condition. The only change that we will make to the code is to modify the loop body, as below.

```
while (NOT(nextNum == 0))
{
  soFar <-- soFar + nextNum
  nextNum <-- nextNum - 1
}
```

With this modified code, after the k th execution of the loop we have the following equations.

$$\begin{aligned}soFar_k &= soFar_{k-1} + nextNum_{k-1} \\nextNum_k &= nextNum_{k-1} - 1\end{aligned}$$

Thus

$$\begin{aligned}soFar_k + INTSUM(nextNum_k) &= soFar_{k-1} + nextNum_{k-1} + INTSUM(nextNum_{k-1} - 1) \\&= soFar_{k-1} + INTSUM(nextNum_{k-1}).\end{aligned}$$

(For the last step, note that, from the specification of *INTSUM*, we have that $n + INTSUM(n - 1) = INTSUM(n)$ for any integer $n > 1$, and in particular for n equal to $nextNum_{k-1}$.)

The above argument shows that $soFar + INTSUM(nextNum)$ is a loop invariant for the modified version of *sumTry*. We have already shown in part (b) that the proposed loop invariant condition (LIC) holds upon entry to the loop (the modified code does not alter this). Hence we have shown that (LIC) is indeed a loop invariant condition for the modified code. The argument given in part (c) above holds for the modified code, so we can deduce that the modified version of *sumTry* is a correct implementation of the function *INTSUM*.

Index

- clean room, 4
- full binary tree, 6
- induction hypothesis, 10
- inductive hypothesis, 9
- inductive step, 10
- loop invariant, 23
- loop invariant condition, 23
- mathematical induction
 - basis for, 10
 - inductive step, 10
 - starting condition, 10
 - starting step, 10
 - Version 1, 10
 - Version 2, 11
 - Version 3, 18
- proof of correctness, 23
- proof strategy
 - a formula is a solution of a recurrence system, 15
 - an implementation using a while loop is correct, 25
- recurrence system, 7
 - recursive part, 7
 - solution of, 11
 - initial condition, 7
- sigma notation (for a sum), 17
- starting condition, 7
- stopping condition, 7
- sums
 - results about, 17

